

# SRS DOCUMENT

## TEAM E2

| Contributors Name | Student ID |
|-------------------|------------|
| Trisha Godhasara  | 202403046  |
| Yashvi Pachani    | 202403062  |

| Sr.No | TOPIC                                 | PAGE No |
|-------|---------------------------------------|---------|
| 1     | SRS                                   | 2       |
| 2     | ER Diagram                            | 56      |
| 3     | Relational Table with all Constraints | 57      |
| 4     | Normalized Tables                     | 63      |
| 5     | DDL Script                            | 69      |
| 6     | Queries                               | 77      |

## SRS-INDEX

| Sr.No | TOPIC                                         |
|-------|-----------------------------------------------|
| i) 1. | Purpose                                       |
| 2.a   | Intended Audience                             |
| 2.b   | Reading/Usage                                 |
| 3.    | Product Scope                                 |
| 4.    | Description                                   |
| ii)   | Background Readings                           |
|       | Combined Requirements from Background Reading |
| iii)  | Interview Plan                                |
|       | Interview Summary                             |
| iv)   | Questionnaire                                 |
| v)    | Combined Requirements from Responses          |
| vi)   | Combined Requirements from Observations       |
| vii)  | Fact Finding Chart                            |
| viii) | Final Combine Requirements                    |
| ix)   | User Classes and Characteristics              |
| x)    | Operating Environment                         |
| xi)   | Product Functions                             |
| xii)  | Privileges                                    |

|       |                      |
|-------|----------------------|
| xiii) | Assumptions          |
| xiv)  | Business Constraints |

### **Purpose:**

The **Placement Database** is designed to centralize and organize all the key data involved in the university placement process.

It acts as the backbone of the system that stores detailed information about students, companies, job offers, placement drives, and interviews. The main goal of the database is to make the placement process more efficient by reducing data redundancy, ensuring data integrity, and providing real-time access to relevant information for all stakeholders—students, placement officers, and companies.

This database streamlines the management of placement data, allowing placement officers to track student progress, schedule interviews, and generate reports quickly.

For students, it provides a single platform to view job opportunities, track applications, and manage documents. Ultimately, the database supports a seamless flow of information, ensuring that no data is lost, and all participants in the process are on the same page.

---

**Here are few challenges that are faced :**

### **Challenge 1: Dispersed and Unstructured Data**

#### **Problem:**

In many universities, placement-related information is often scattered across various platforms—such as spreadsheets, emails, and paper records. This unorganized data makes it difficult to track student progress, manage company details, and generate accurate reports.

#### **Solution:**

The Placement Database solves this problem by centralizing all placement data into a relational database. Data is organized into structured tables (e.g., Students, Companies, Job Offers, Interviews) with clearly defined relationships. For example, each student's details are linked to

their interview schedules and job offers, creating a unified data structure that's easy to query and update.

**Impact:**

Centralizing the data ensures that all information is stored in a single source of truth. Placement officers can access and update records without confusion or risk of error. This improves data retrieval speed and ensures everyone has access to the most up-to-date information.

**Challenge 2: Data Redundancy and Inconsistency**

**Problem:**

Without a proper database structure, the same information might be entered multiple times across different platforms, leading to data redundancy and inconsistencies. For example, a student's personal details may be stored in several places, and any updates to that information might not be reflected everywhere.

**Solution:**

The Placement Database uses data normalization techniques to eliminate redundancy. Normalization ensures that each piece of information is stored only once, and any references to that data (such as a student's name or company details) are linked through **foreign keys**. This prevents duplication and ensures that any updates made to one record are reflected throughout the system.

**Impact:**

By enforcing **referential integrity** and reducing duplication, the database ensures consistent data across all records. Students' and companies' details are accurate, and placement officers can rely on the system for real-time, updated information without worrying about inconsistencies.

**Challenge 3: Inefficient Reporting and Querying**

**Problem:**

Generating reports manually or using unoptimized queries in non-database systems is time-consuming and prone to errors. For example, placement officers might struggle to track the status of student placements or analyze which companies are hiring the most candidates.

**Solution:**

The **Placement Database** enables **fast querying** through **SQL** (Structured Query Language). By organizing data in tables with **indexes** and optimizing query performance, the database allows placement officers to quickly generate reports on key metrics like the number of placements, student progress, or company participation. Predefined queries can help track how many students have been placed, which companies attended placement drives, or which job offers have been extended.

**Impact:**

Efficient querying speeds up the entire process of generating reports. Placement officers can analyze placement trends, monitor student success, and evaluate which companies are actively participating—all with a few simple commands. This leads to **data-driven decision-making** and improves the overall effectiveness of the placement process.

**Challenge 4: Data Security and Access Control****Problem:**

Sensitive data, such as student personal information, job offers, and interview feedback, needs to be protected. In traditional systems, ensuring that only authorized individuals can access certain types of data is often difficult to implement.

**Solution:**

The Placement Database incorporates **role-based access control (RBAC)**, ensuring that only authorized users (like placement officers) can access or modify specific records. For instance, students can view their own details and job applications, while placement officers have full access to student and company data. Additionally, sensitive data is encrypted to ensure privacy and security.

**Impact:**

This enhanced security ensures that personal information and sensitive documents are protected. Role-based access control provides a structured way to manage permissions, reducing the risk of unauthorized access and ensuring compliance with data privacy regulations.

## **Challenge 5: Data Isolation**

### **Problem:**

When multiple users (students, placement officers, recruiters) access and update data simultaneously, there is a risk of dirty reads, phantom reads, lost updates, or inconsistent reports. For example, a recruiter might view an outdated candidate list because another admin's changes are not yet committed.

### **Solution:**

Implement proper transaction isolation levels to control concurrent access. Use row-level locking or optimistic concurrency control to avoid conflicts. Regularly monitor for deadlocks and optimize queries to reduce lock contention.

### **Impact:**

- Provides **accurate and up-to-date data** to all users.
- Improves system reliability and enhances user confidence, especially during high-traffic events like mass registrations or interview scheduling

## **Challenge 6: Crash Recovery**

### **Problem:**

Unexpected crashes (due to power failure, network outage, or server malfunction) may lead to data loss or corruption. For example, if a student registers for a placement drive and the crash occurs mid-transaction, the database may have incomplete or inconsistent data, causing confusion later.

**Solution:** The system should keep a log of every change so that it can finish any incomplete work after restarting. Regular automatic backups should be taken and stored safely (on a server or cloud). Use **atomic transactions** so that either all steps of a process are completed or none at all. If a crash happens, the backup can be quickly restored so work continues smoothly. This avoids starting everything from scratch.

### **Impact:**

- Ensures data integrity and consistency even after failures.
- Minimizes downtime and prevents loss of important placement data.
- Builds trust among students, companies, and administrators that the system is reliable during peak placement activity.

## **Conclusion**

In conclusion, the **Placement Database** provides a structured, efficient solution to the challenges faced by universities during the placement process. By centralizing all placement data, eliminating redundancy, optimizing reporting, and ensuring data security, the database helps improve the overall management of student placements. It makes data easily accessible, accurate, and up-to-date, ensuring a smoother experience for placement officers, students, and companies alike.

By leveraging technologies like **SQL**, **normalization**, and **role-based access control**, the database ensures that all stakeholders can track placement progress, manage opportunities, and make informed decisions based on reliable data. This makes the entire placement process more streamlined, transparent, and efficient.

---

## **Intended Audience of a University Placement Database**

### **1. Students (UG, PG, PhD, Masters)**

- Final-year and pre-final-year students preparing for campus placements.
- Freshers and juniors exploring what kinds of companies visit the campus.
- Students aiming for higher studies, since placement trends highlight industry relevance of their program.

### **2. Placement Cell / Career Services Team**

- To monitor company participation and package trends.
- To maintain updated records for inviting recruiters.
- To evaluate placement training effectiveness (aptitude, coding, GD, interview sessions).
- To design workshops aligned with industry requirements.

### **3. Faculty Mentors / Department Coordinators**

- To guide students in choosing electives, projects, and internships that match market demand.
- To showcase department-specific placement performance.
- To mentor students based on past recruitments (e.g., which companies prefer which specializations)

#### **4. Recruiters / Companies**

- To get a snapshot of the university talent pool and past placement history.
- To benchmark with their recruitment at other institutions.
- To understand diversity of hires (departments, gender, skill levels).
- To build trust in continuing or expanding recruitment drives.

#### **5. University Administration & Management**

- To measure institutional performance and reputation.
- To highlight achievements in brochures, NIRF/NAAC reports, and accreditation submissions.
- To attract prospective students and parents during admissions.
- To justify curriculum reforms in line with placement outcomes.

#### **6. External Agencies (Accreditation, Media)**

- Accreditation bodies (NAAC, NBA, ABET) use placement records for quality benchmarking.
- Media outlets for publishing annual placement highlights.

#### **7. Alumni**

- To share job openings and refer students for roles in their companies.
- To mentor students and provide career guidance based on real-world experience.
- To stay connected with the institution's placement activities and support its growth.
- To help expand the recruiter network by introducing new hiring partners.

#### **Reading / Usage Suggestions**



The placement database is intended for use by a wide range of stakeholders, and the way it is read and interpreted depends on the user group. Since the database provides both raw records and summarized reports, it is important that readers focus on the aspects most relevant to their role.

- **For Students**

- Look for company-wise details (roles, packages, eligibility, skills tested).
- Analyze sector trends (e.g., IT vs Core vs Consulting).
- Use it to plan preparation strategies (aptitude, coding, interviews, projects).

- **For Placement Cell / Faculty**

- Track year-on-year statistics (number of companies, offers, highest/median packages).
- Identify skill gaps and recommend training workshops.
- Match student interests with recruiter demands.

- **For Recruiters**

- Understand diversity of candidates placed (departments, backgrounds).
- Benchmark recruitment outcomes with other institutions.

- **For Alumni**

- Alumni may use the database to stay informed about the university's current placement trends and compare present statistics with their own placement experiences.
- The database can help them explore opportunities for networking or collaboration with recruiters.
- Alumni who are now part of the industry may use the data to strengthen engagement with the university.

- **For Administration**

- Read it as a performance indicator for annual reports.
  - Use for marketing to prospective students/parents.
-

## **Product Scope**

The placement database is meant to be a one-stop system for everything related to student placements in the university. It will bring together all the key details in one place—student profiles, academic records, skills, company information, job roles, eligibility criteria, and salary packages—while also keeping track of who got placed where, including both internships and full-time offers.

Beyond storing data, the database will be useful for generating insights. It can show year-wise and department-wise trends, highlight which companies visit most often, and give a clear picture of salary ranges and placement percentages. This makes it helpful for students who want to plan their preparation, for faculty who guide them, for recruiters who assess their hiring outcomes, and for the university administration when preparing reports or accreditation documents.

The focus of the system is only on placements, so it will not deal with unrelated areas like faculty payroll, examination results, or alumni career tracking after graduation. What it will deliver, however, is simple and valuable: a secure and well-organized database, exportable reports and summaries, and easy-to-read dashboards that make placement trends clear at a glance. In short, the placement database is designed to make the entire placement process more organized, transparent, and useful for everyone involved.

Its key deliverables include:

- A secure and structured database of placement data.
- Exportable reports and annual summaries for administration and accreditation.
- Interactive dashboards to visualize placement trends and outcomes.

In essence, the placement database will provide a comprehensive yet focused solution for managing and presenting placement-related information in a way that benefits all stakeholders.

---

## **Description of the Placement Database**

Universities have a Placement Office (TPO – Training & Placement Office) that plays a critical role in helping students secure jobs and internships. This office collaborates with student volunteers and companies to organize placement drives and internship opportunities. However, as these activities grow in scale and complexity, managing them using traditional, manual methods (like spreadsheets and paper records) becomes increasingly difficult. This is where a comprehensive Placement Database comes into play.

The Placement Database is a relational database management system (RDBMS) designed to store and manage all information related to the placement process. It acts as the backbone for managing students' placement details, companies' job offers, placement drive schedules, and interviews. The purpose of this database is to centralize the data and make it easily accessible, eliminating the inefficiencies caused by scattered records and manual tracking.

The system is not just a data repository but is built with specific functionalities that enhance the placement process. It integrates key features like tracking student applications, managing company profiles, scheduling interviews, recording offers made by companies, and maintaining a history of each placement drive. This ensures that placement officers and students can easily retrieve the necessary information at any point in time.

Given below is a brief description of the critical components that the Placement Database will offer:

1. **Student Records:** The database will store detailed information about students, including personal details (e.g., name, contact information), academic background (e.g., GPA, skills), and records of the companies they have applied to. The system also tracks which students have attended which interviews and the status of each job offer.
2. **Company Profiles:** The database will manage detailed company profiles, including company names, industries, job roles available, eligibility criteria, and requirements. Each placement drive will be associated with a particular company, making it easy for placement officers to manage multiple placement events.
3. **Placement Drives & Schedules:** Placement drives are central events where multiple companies visit the university to hire students. The database will help track each placement drive, including the dates, locations, participating companies, and the students

involved. This ensures that placement officers can coordinate schedules and monitor which students have attended which drives.

4. **Job Offers & Interview Status:** The database will also maintain information about job offers made by companies, including job titles, salary details, and the interview process for each student. This enables placement officers to track which students have received offers and manage post-offer communication between students and companies.
  5. **Reports & Analytics:** The system will allow for the generation of customized reports on placement statistics, trends, and outcomes. Placement officers can access data like the total number of students placed, average salary offered, number of students per company, and more.
-

### **Background Readings**

|  | S.No | Title/Source                                                                           | Type    | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|--|------|----------------------------------------------------------------------------------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | 1    | <b>Database System Concepts by Abraham Silberschatz, Henry Korth, and S. Sudarshan</b> | Book    | <ul style="list-style-type: none"><li>➤ This textbook gave a strong foundation in database design, normalization, and entity-relationship modeling.</li><li>➤ It was particularly useful in planning the placement database structure and ensuring efficient storage of student and company information.</li><li>➤ In addition, the book provided insights into indexing, query optimization, and transaction management, which are essential for building a reliable and scalable database.</li><li>➤ Its practical examples made it easier to translate theoretical concepts into the design of the placement database system.</li></ul> |
|  | 2    | <b>NAAC (National Assessment and Accreditation Council) Official Website</b>           | Website | <ul style="list-style-type: none"><li>➤ Defines <u>Criterion V</u> – Student Support &amp; Progression, which includes placement metrics such as the percentage of outgoing students placed and progression to higher education.</li><li>➤ Specifies that institutions must maintain records of placement data for the last five years, including offer letters and placement cell reports.</li><li>➤ Emphasizes the need for accurate, year-wise placement tracking—this influences system requirements such as reporting flexibility, data retention, and export capabilities for accreditation submissions</li></ul>                    |

|  |   |                                                                                                                                                                                                            |                        |                                                                                                                                                                                                                                                                                                                                |
|--|---|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  |   |                                                                                                                                                                                                            |                        |                                                                                                                                                                                                                                                                                                                                |
|  | 3 | <b>IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998)</b><br><a href="https://standards.ieee.org/ieee/830/1222/?utm">https://standards.ieee.org/ieee/830/1222/?utm</a> | Standard/<br>Guideline | <ul style="list-style-type: none"> <li>➤ This guideline helped in understanding the <u>formal structure</u> of an SRS, including key sections like product scope, intended audience, and functional requirements.</li> <li>➤ It ensured that the SRS for the placement database followed <u>industry standards</u>.</li> </ul> |

|  |   |                                                                                                      |         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|--|---|------------------------------------------------------------------------------------------------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | 4 | <b>Postgres-SQL Website</b><br><a href="https://www.postgresql.org/">https://www.postgresql.org/</a> | Website | <ul style="list-style-type: none"> <li>➤ <u>Reliability &amp; Scalability</u>: PostgreSQL is known for its enterprise-grade reliability, making it suitable for a university placement database where data accuracy is critical.</li> <li>➤ <u>Cost-Effective</u>: It is open-source and free to use, helping universities avoid expensive database licensing fees.</li> <li>➤ <u>ACID Compliance</u>: Guarantees atomicity, consistency, isolation, and durability ensuring correct data even with multiple simultaneous users.</li> <li>➤ <u>Advanced Query Support</u>: Offers indexing, joins, and full-text search useful for quick student search, filtering job postings, and generating reports.</li> <li>➤ <u>Security Features</u>: Provides strong authentication, SSL encryption, and row-level security to protect sensitive student and recruiter data.</li> <li>➤ <u>Community &amp; Support</u>: Has a large global community and frequent updates, ensuring long-term stability and continuous improvement.</li> <li>➤ <u>Cross-Platform &amp; Extensible</u>: Runs on major operating systems and supports custom extensions, making it flexible for future enhancements.</li> </ul> |
|--|---|------------------------------------------------------------------------------------------------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|  |   |                                                                                                                                                                                                                                                                        |                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|--|---|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | 5 | <b>NIRF<br/>(National Institutional Ranking Framework) India Reports</b>                                                                                                                                                                                               | Report/<br>Website | <ul style="list-style-type: none"> <li>➤ The website emphasized how accurate and well-maintained placement records play a vital role in accreditation processes and institutional assessments, such as those conducted by bodies like NAAC or NIRF.</li> <li>➤ These records not only demonstrate the university's ability to support student employability but also serve as a benchmark for comparing institutional performance with peers.</li> <li>➤ By highlighting the <u>direct link</u> between placement outcomes and quality ratings, the reading made it clear why a structured placement database is not just a convenience but a strategic necessity for universities.</li> <li>➤ It helps institutions maintain credibility, attract prospective students, and build stronger connections with industry stakeholders.</li> </ul> |
|  | 6 | <b>DA-IICT Placement Website</b><br><br><a href="http://placement.daiict.ac.in/Home/placementPolicy"><u>http://placement.daiict.ac.in/Home/placementPolicy</u></a><br><br><a href="https://www.daiict.ac.in/placements"><u>https://www.daiict.ac.in/placements</u></a> | Website            | <ul style="list-style-type: none"> <li>➤ The official <u>Placement Cell portal of DA-IICT</u> provides comprehensive information about the institute's placement policies, organizational structure, and engagement with recruiters.</li> <li>➤ It also highlights placement statistics, eligibility guidelines, and details of the placement cycle, making it a valuable reference for students and companies alike.</li> <li>➤ Reviewing this website offered useful insights into how placement activities are organized and documented at the institutional level, and it helped in shaping the understanding of what kind of data and processes are important to capture in a placement database.</li> </ul>                                                                                                                              |



|  |   |                                                                                  |         |                                                                                                                                                                                                                                                                                                                                                       |
|--|---|----------------------------------------------------------------------------------|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | 7 | <b>Data Privacy and Protection Guidelines (India's DPDP Act, 2023)</b>           | Article | <ul style="list-style-type: none"> <li>➤ Placement data includes personal information (name, email, resume).</li> <li>➤ The system must follow <u>data privacy compliance</u>: controlled access, student consent, and secure storage.</li> <li>➤ Reinforces the need for role-based access control (students, placement cell, recruiters)</li> </ul> |
|  | 8 | <b>Industry Placement Trend Reports (e.g., NASSCOM, LinkedIn Economic Graph)</b> | Article | <ul style="list-style-type: none"> <li>➤ Gives insights into skill demand trends, guiding what data to collect (skills, certifications, internships).</li> <li>➤ Helps placement cell align training modules with industry expectations.</li> <li>➤ Encourages tracking intern-to-full-time conversion rates.</li> </ul>                              |

## **Combined Requirements from Background Reading**

### **1. Database Design & Functionality**

- Must support entity–relationship modeling, normalization, indexing, and query optimization (from Database System Concepts).
- Ensure transaction management for reliability (atomicity, consistency, isolation, durability).
- Provide efficient student search, job filtering, and report generation features (from PostgreSQL site).

### **2. Regulatory & Accreditation Compliance**

- Maintain year-wise placement data for at least five years (NAAC requirement).
- Store offer letters, placement reports, and student progression records.
- Provide exportable reports formatted for NAAC & NIRF submissions.
- Placement outcomes must be accurately tracked and comparable across years/institutions.

### **3. Software Engineering Standards**

- Follow IEEE Std 830-1998 SRS structure, ensuring clear definition of:
  - Product scope
  - Intended audience
  - Functional and non-functional requirements

### **4. System Technology & Infrastructure**

- Use an open-source, reliable, scalable RDBMS (e.g., PostgreSQL).
- Must be cross-platform and extensible for future enhancements.
- Ensure enterprise-grade reliability and community support for long-term use.

### **5. Placement Process Support**

- Capture placement policies, eligibility criteria, organizational structure, and recruiter engagement details (from DA-IICT portal).
- Store placement statistics and cycle details for institutional benchmarking.
- Enable tracking of internships and conversion to full-time jobs (from industry reports).

## **6. Security & Privacy**

- Comply with India's DPDP Act (2023).
- Enforce role-based access control (students, recruiters, placement officers).
- Ensure student consent for data usage and secure storage.
- Use encryption (SSL) and authentication mechanisms.

## **7. Industry & Student-Centric Features**

- Record skills, certifications, internships of students to align with recruiter needs.
- Provide insights into industry demand trends for guiding training modules.
- Allow placement cell to generate trend reports and analytics.

## **Interview Plan**

**PLACEMENT CELL :** Interview Plan System

**Project-** E2/46/62/MnC

**Participants:** Rishit Unadkat(Convener)  
Kartavya Akbari(Deputy Convener)  
Ms Deepali Sharma(Assistant Manager)

**Date:** 2 Sep 2025

**Time:** 9 pm

**Duration:** 2.5hr

**Place:** DAU

### **Purpose of Interview:**

Preliminary meeting to understand the placement cell's current processes, requirements, and challenges faced.

### **Agenda:**

- ☐ Challenges faced in the current placement process
- ☐ Data management of students and companies
- ☐ Interview scheduling and tracking
- ☐ Communication between students, companies, and the placement cell
- ☐ Ideas for improving the placement system
- ☐ Follow-up actions

### **Documents to be brought to the Interview:**

Challenges and feedback log, list of companies, some of the student's CV, sample forms that company and students currently use.

---

## **Interview Summary**

**ITSolutions:** Interview Summary

**System :** Placement Database

**Project Reference :** E2/46/62/MnC

**Participants:** Rishit Unadkat(Convener-Placement Cell-DAU)  
Kartavya Akbari(Deputy Convener-Placement Cell-DAU)  
Ms Deepali Sharma(Assistant Manager)

**Duration :** 2.5 hr

**Place:** DAU

### **Combined Requirements gathered from Interview:**

- Current Process in DAU for placement
- The role of Placement Officer
  - ❖ Acts as the central node, managing all communication between companies, students, and college departments.
  - ❖ Personally oversees data entry, schedule coordination, and report generation, creating a single point of failure.
  
- How they manage back-ups and crash recovery
  - ❖ Basic & Risky. Likely manual saves to a local computer or a shared drive (like Google Drive).
  - ❖ No formal, automated backup or disaster recovery process exists, risking significant data loss. In some of the colleges this takes place
  
- Issues/Mistakes student do more often
  - ❖ Missing deadlines for applications and confirmations.

- ❖ Providing incorrect data in their forms (typos in GPA, contact info) and applying to companies whose eligibility criteria they don't meet.

→ Problem faced in managing offer letters

- ❖ Manual & Redundant. The officer likely downloads emails from companies and manually updates the master student tracking spreadsheet.
- ❖ This creates a delay in reflecting the "Placed" status and centralizing the document

→ Dispersed alumni data

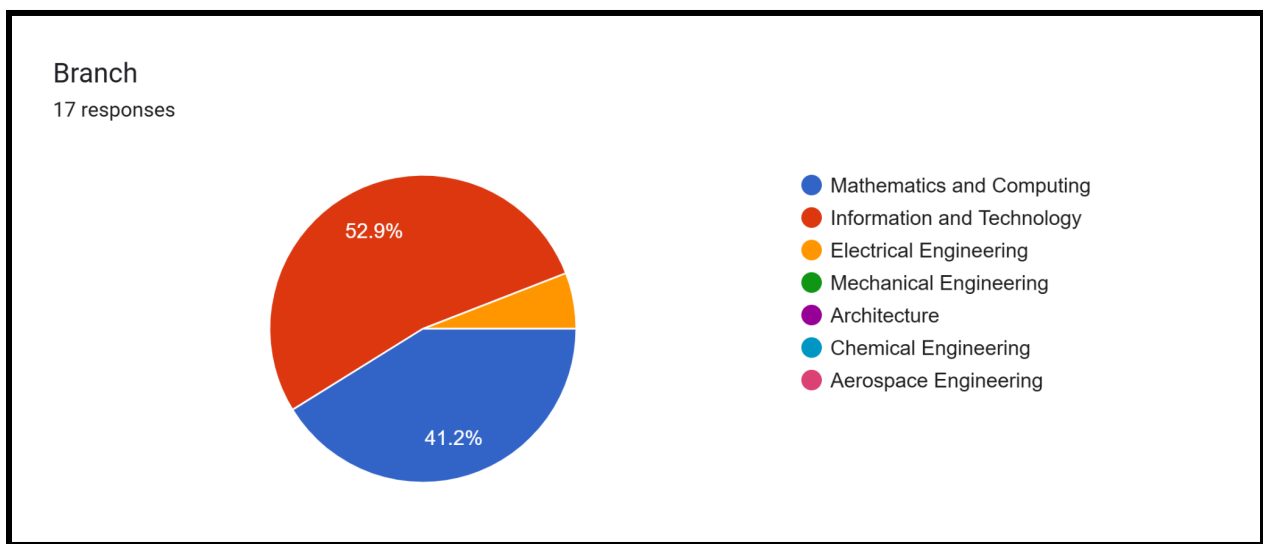
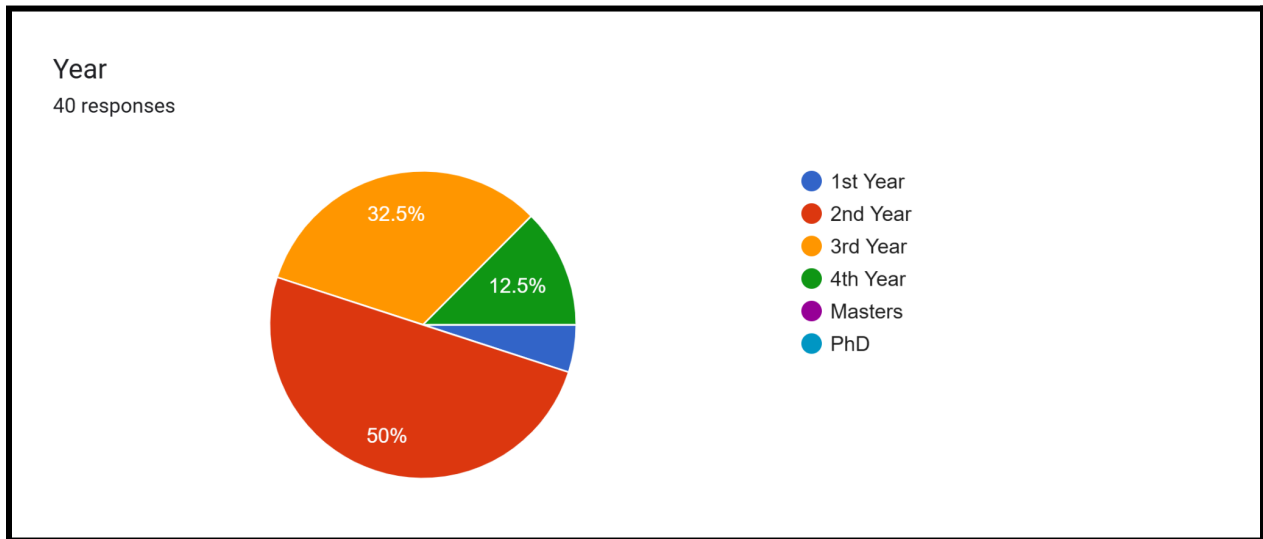
- ❖ No Formal Database: Alumni contact information (especially updated workplace details) is likely scattered—in personal contacts, old spreadsheets, or LinkedIn. It is not integrated into the active placement management system.
-

## Questionnaire

Google Form

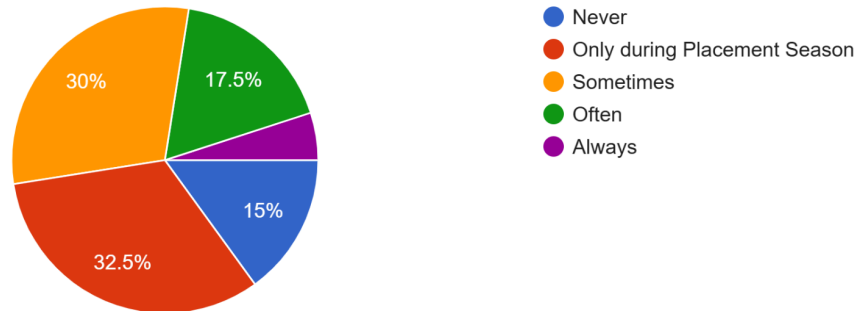
[https://docs.google.com/forms/d/e/1FAIpQLSdOkPc2D1i9W4R8FV-O\\_o3NFGz6lcE2zPV8-U8bzYmtXzrO4A/viewform?usp=sharing&ouid=101434862020722217105](https://docs.google.com/forms/d/e/1FAIpQLSdOkPc2D1i9W4R8FV-O_o3NFGz6lcE2zPV8-U8bzYmtXzrO4A/viewform?usp=sharing&ouid=101434862020722217105)

**Category:STUDENTS**



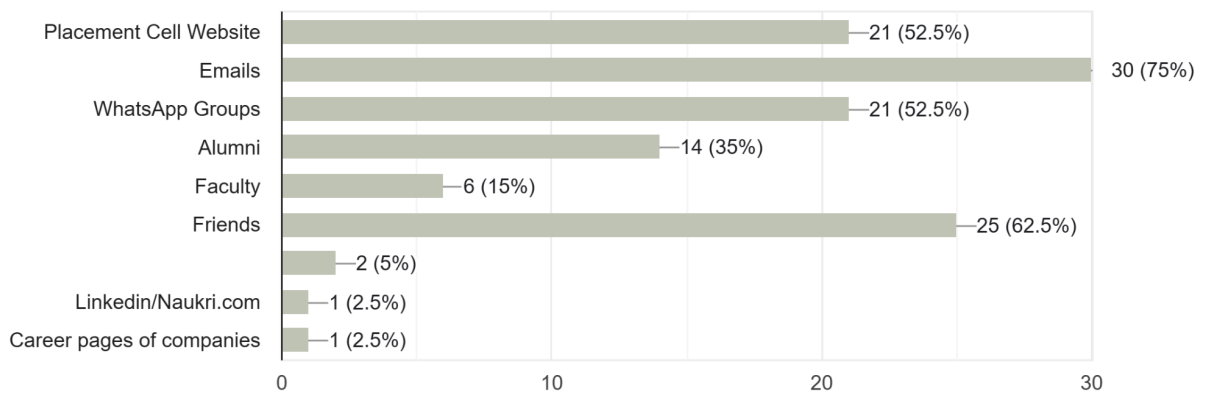
### How often do you use placement-related resources (portals, notices, recruiter info)?

40 responses



### Which platforms do you currently use most for placement-related updates?

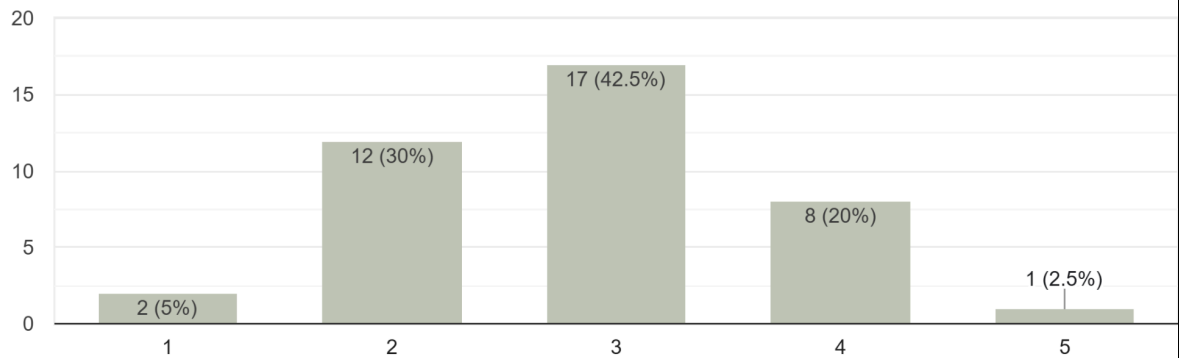
40 responses





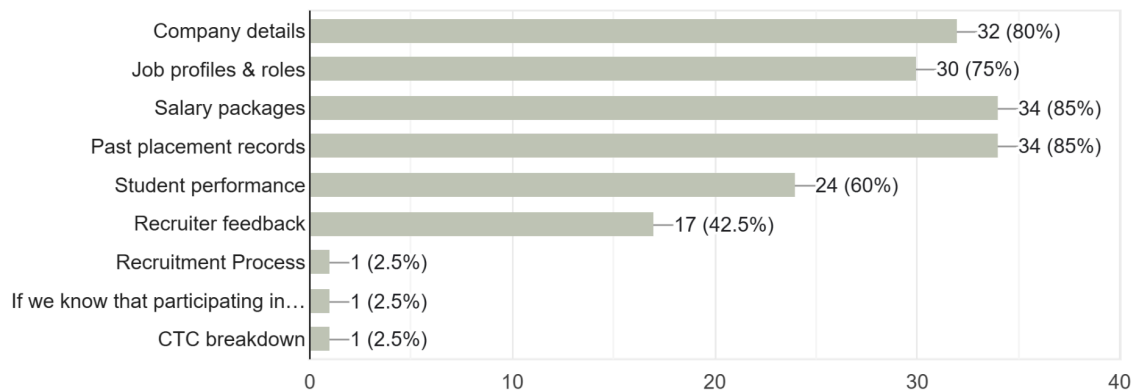
### How satisfied are you with the current placement information system?

40 responses



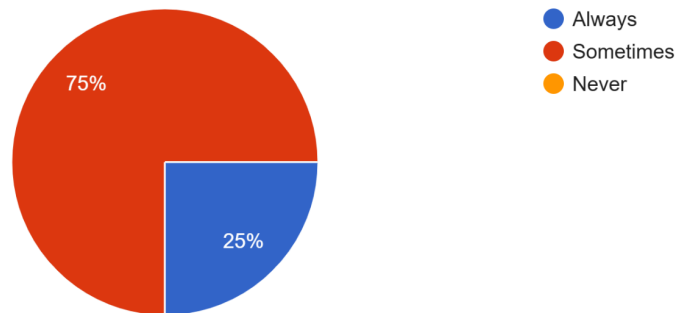
### What kind of placement information do you usually look for?

40 responses



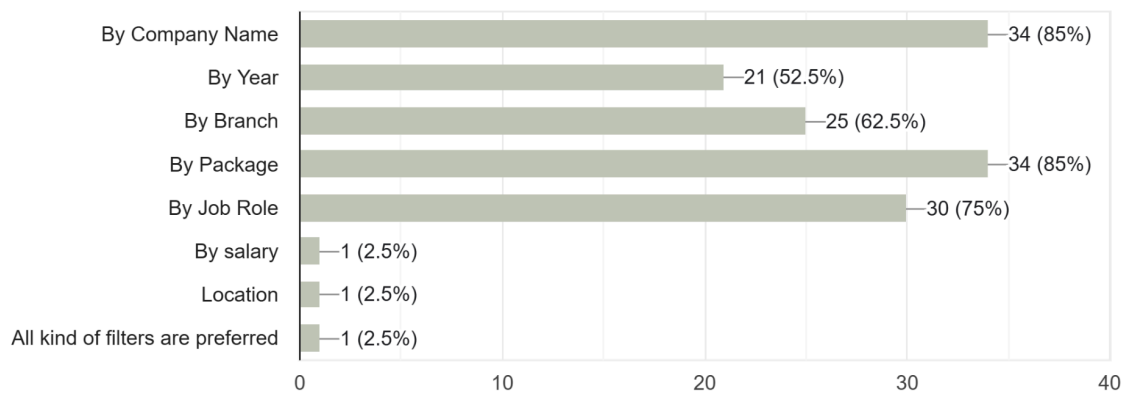
How often do you find it difficult to access reliable placement information?

40 responses



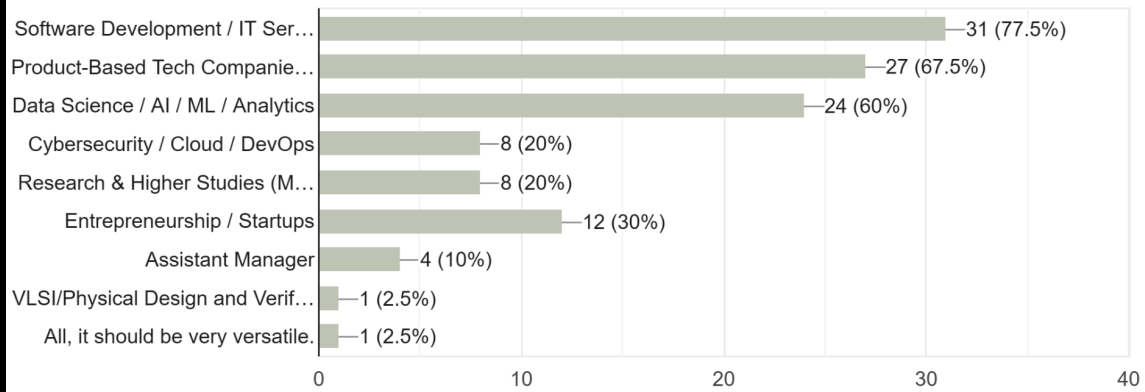
Which search/filtering options would you like to see?

40 responses



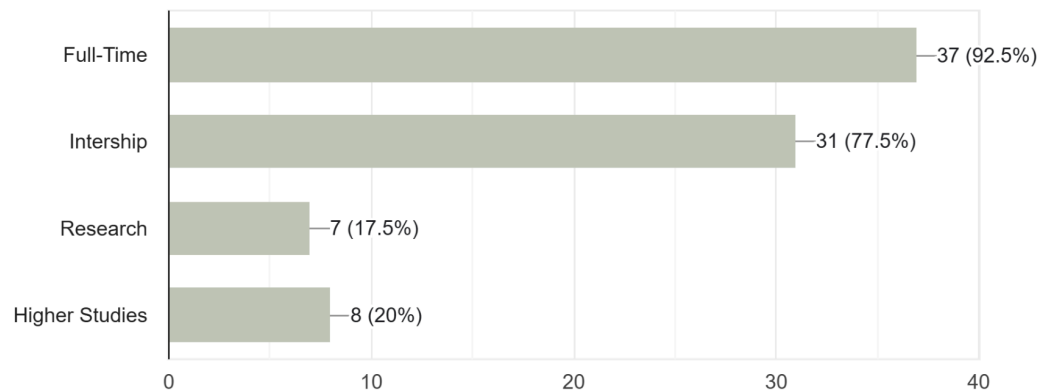
### What sectors or roles are you most interested in?

40 responses



### Interested in

40 responses



What difficulties do you currently face in the placement process?

40 responses



**CATEGORY: COMPANY**

## Company

Company Name

0 responses

No responses yet for this question.

How often does your organization recruit from our institution?

0 responses

No responses yet for this question.

Which types of roles do you usually offer?

0 responses

No responses yet for this question.

Which levels of recruitment do you primarily conduct?

0 responses

No responses yet for this question.

What student information is most valuable for your recruitment process?

0 responses

No responses yet for this question.

Would you prefer student profiles to follow a standardized format?

0 responses

No responses yet for this question.

What additional data or reports would you find useful in a placement database?

0 responses

No responses yet for this question.

Who from your organization should ideally have access to the placement database?

0 responses

No responses yet for this question.

Which method of accessing the database would you prefer?

0 responses

How important is it for you to view placement statistics of previous years?

0 responses

No responses yet for this question.

Which format would you prefer for placement statistics?

0 responses

No responses yet for this question.

What difficulties do you currently face during the recruitment process at our institution?

0 responses

No responses yet for this question.

What features would make the placement database more recruiter-friendly?

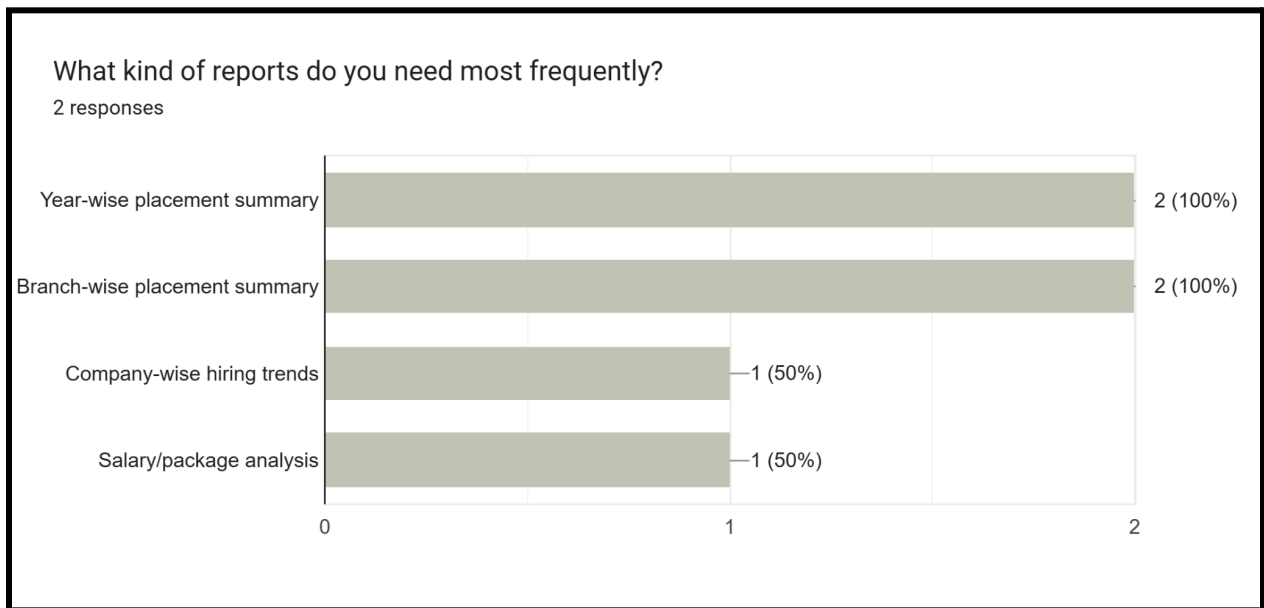
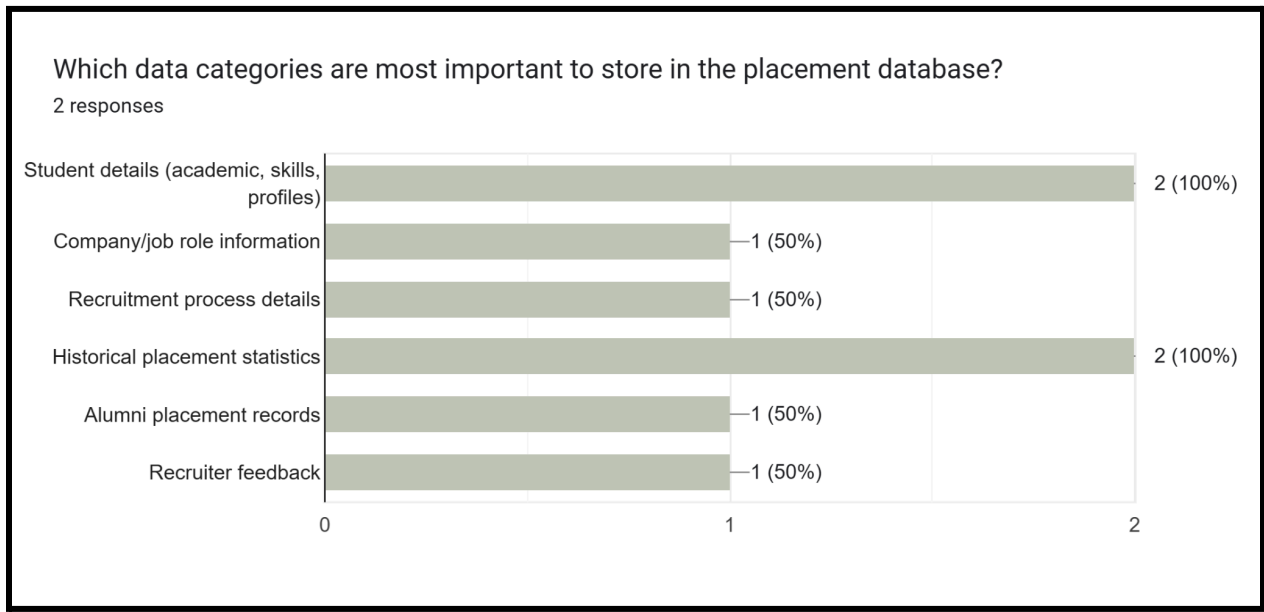
0 responses

No responses yet for this question.

Please share any additional suggestions or expectations you may have:

0 responses

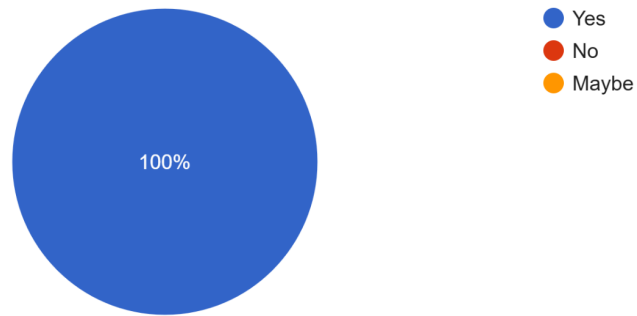
CATEGORY: PLACEMENT CELL MEMBER





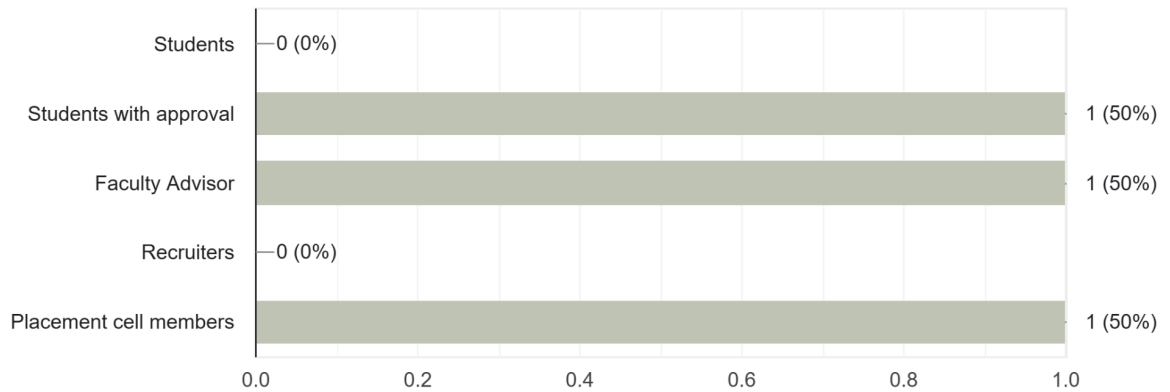
Would you like the database to support automated report generation?

2 responses



Who should have access to edit or update placement records?

2 responses



What additional features or improvements would you like to see in the placement database?

2 responses

i would like the students to see their own progress i.e can check which type of interviews they have passes and what is stopping them to achieve their dream job

There can be individual or mass feedback system by company for students. Additionally offer letters can also be added in database.

### **Combined Requirements from Responses**

Based on the questionnaire conducted and the summarized responses (graphs and short answers) from the Google Form , the following **combined requirements** have been identified for the Placement Database Management System:

#### **1. Placement Resource Accessibility**

- Students require a **centralized repository** of placement-related resources that can be accessed at any time.
- The repository should contain:
  - **Aptitude and coding practice materials** (sample questions, mock tests).
  - **Company profiles** with past recruitment patterns, eligibility criteria, salary packages, and job roles.
  - **Interview preparation materials** (common interview questions, HR questions, technical guides).
  - **Previous year placement data** for reference.

- The system must ensure that the resources are well-organized, categorized, and searchable to help students quickly find relevant content.
- Some students indicated that they **rarely use existing resources**, so the platform should be made more **interactive, engaging, and regularly updated** to encourage usage.

## 2. Communication and Notifications

- One of the strongest requirements that emerged is the need for a **reliable communication system**. Students rely on multiple channels such as WhatsApp groups, email, and the placement website.
- The new system should integrate **real-time notifications** and send alerts via:
- Backup communication methods must be implemented to ensure that **no critical information is missed** by students.

## 3. Interview Scheduling and Tracking

- Students expressed the need for a **transparent and structured scheduling system** for interviews and placement drives.
- Features required include:
  - A **calendar view** showing upcoming placement activities.
  - The ability to check **individual interview slots**.
  - Automated **reminders** for interview dates and deadlines.
  - Tracking of **attended vs. missed interviews** to help students manage their progress.
- Placement officers should be able to update and manage interview schedules centrally, ensuring consistency and clarity across all users.

#### 4. User Experience and Accessibility

- Students require a **simple, user-friendly dashboard** with clear navigation.
- The system should be accessible across devices, especially **mobile phones**, since most students check updates on WhatsApp or email via their phones.
- **Search and filter features** must be provided so that students can easily look up companies, deadlines, or specific resources.
- The interface should support **personalized views** (e.g., highlighting only the companies a student is eligible for).

#### 5. Placement Cell Features and Analytics

- The system should generate **reports and analytics**, such as:
  - Student participation in mock tests and training sessions.
  - Company-wise and student-wise placement statistics.
  - Resource usage tracking to identify what students are finding helpful.
- Graphical summaries (charts, dashboards) should be available for quick insights.
- A **feedback module** should allow students to share their experiences, which will help the placement cell improve future placement activities.

#### 6. Overall System Requirements

- **Centralized platform** combining resources, scheduling, and communication in one place.
- **Role-based access** (different dashboards for students, placement officers, and administrators).
- **Scalability** to handle large volumes of data (student records, company records, past data).
- **Security features** to ensure student data is protected and accessible only to authorized users.

- A **sustainable update mechanism** so the system stays relevant year after year.
- 

## **Conclusion**

The combined requirements gathered from the questionnaire responses highlight the need for a **comprehensive placement management system** that not only stores resources but also actively engages students, streamlines communication, and empowers placement officers with data insights. Such a system would make the placement process more **efficient, transparent, and student-friendly**, ultimately benefiting both the students and the placement cell.

---

## **List of Combined requirements from Observations**

### **1. Data Storage & Management**

- Student information must be stored (academic records, skills, certifications, projects, extracurriculars).
- Company information must be stored (job roles, eligibility criteria, selection process, offered packages).
- Placement history should be maintained (year-wise, branch-wise, company-wise, alumni records).
- Recruiter feedback and remarks should be captured.
- Database should allow bulk upload/import of records (e.g., Excel/CSV).

### **2. Accessibility & Roles**

- Role-based access control must be implemented:
  - **Students** → Can view placement details
  - **Placement Cell** → Full access to create, update, and manage all records.
  - **Faculty** → Limited access for monitoring and verification.

- **Companies/Recruiters** → Access to standardized student profiles and reports.
- Both **web-based portal** and **mobile-friendly access** are required.

### 3. Search, Filter, and Reporting

- Search and filter options (by company, year, branch, package, role).
- Generation of reports (year-wise, branch-wise, company-wise, salary analysis).
- Placement statistics should be available in multiple formats (tables, charts, dashboards).
- Recruiters should be able to access standardized student profiles easily.

### 4. User Experience & Usability

- Simple, intuitive, and user-friendly interface.
- Support for graphical dashboards (for trends and quick insights).
- Notifications and reminders for upcoming deadlines, recruitment events, or profile updates.
- Data should be exportable for official reports (PDF, Excel).

### 5. Security & Data Integrity

- Student profile edits must require approval from Placement Cell.
- Secure login system for all stakeholders.
- Protection of sensitive data (e.g., salary packages, personal details).
- Audit trails to track updates and changes.

### 6. Additional Observations

- Students expect **easy access to reliable placement information**.
- Historical placement trends are important for both recruiters and institutional reporting.
- To strengthen institutional reporting and accreditation processes, the Placement Database should also store **official placement-related letters and documents** such as:
  - Offer letters from companies
  - Confirmation letters for internships and full-time roles
  - Company MoUs and correspondence with recruiters

- Placement confirmation reports
- 

### **Fact Finding Chart**

| Objective                                                   | Technique          | Subject(s)                             | Time    |
|-------------------------------------------------------------|--------------------|----------------------------------------|---------|
| To get the background of the placement system               | Background reading | Books, websites, articles, reports     | 1 day   |
| Involvement of Placement Office                             | Interview          | Assistant Manager                      | 20 mins |
| Scheduling of company interviews                            | Interview          | Placement cell members                 | 0.5 hr  |
| Managing the Online Assessment Test                         | Interview          | Placement cell members                 | 10 mins |
| General view of offers that company brings                  | Interview          | Placement cell members, Student's view | 15 mins |
| Student Issues                                              | Questionnaire      | Students                               | 20 mins |
| Back-up and recovery                                        | Interview          | Placement cell Members                 | 15 mins |
| Interview and resume shortlisting                           | Interview          | Placement cell Members                 | 20 mins |
| Communication between students placement cell and companies | Interview          | Placement cell Members, Admin          | 0.5 hr  |
| Search and filtering the students and companies need to see | Questionnaire      | Students, Company                      | 1.5hr   |
| Difficulties faced in current system                        | Questionnaire      | Students, Placement cell               | 30min   |

## **Final Combine Requirements**

### **1. Data Storage & Management**

- Store **student information** (academics, skills, certifications, projects, extracurriculars, resumes).
- Store **company/recruiter information** (roles, eligibility criteria, process, packages).
- Maintain **placement history** (year-wise, branch-wise, company-wise, alumni records).
- Store **official placement documents** (offer letters, internship confirmations, MoUs, placement reports) for **NAAC/NIRF compliance**.
- Enable **bulk upload/import** (Excel/CSV) and **export** (Excel/PDF) of records.
- Support **internship-to-full-time conversion tracking**.

### **2. Placement Resource Accessibility**

- Centralized repository of resources:
  - Aptitude & coding practice materials.
  - Interview prep content (HR, technical).
  - Company profiles with historical hiring trends.
  - Previous year placement data.
- Resources must be **categorized, searchable, and updated regularly**.
- Encourage usage with **interactive features** (quizzes, feedback, engagement tracking).

### **3. Communication & Notifications**

- Real-time notifications for:
  - Placement drives, deadlines, results.
  - Interview scheduling and slot changes.
- Alerts delivered via **portal, email, and mobile push/WhatsApp integration**.
- Backup notification channels to ensure no critical update is missed.



#### 4. Interview Scheduling & Tracking

- **Calendar view** of upcoming activities.
- Automated **interview slot booking and reminders**.
- Track attended vs. missed interviews.
- Placement officers can centrally update schedules.

#### 5. Search, Filter & Reporting

- Search/filter students and companies by year, branch, package, eligibility, skills.
- Generate **placement reports**:
  - Year-wise, branch-wise, company-wise statistics.
  - Salary/package analysis.
  - Recruiter feedback summaries.
- Support **graphical dashboards** (charts, trends, KPIs).
- Data export for **official accreditation reports**.

#### 6. User Roles & Access

- **Students** → View placement details, resources, eligible jobs, interview schedules.
- **Placement Cell/Admins** → Full access to manage records, resources, analytics.
- **Faculty** → Limited monitoring/verification.
- **Recruiters** → Access to standardized student profiles & summary reports.
- Role-based dashboards (personalized views).

#### 7. Security, Privacy & Compliance

- Secure login with authentication and authorization.
- **Role-based access control** for all stakeholders.
- Comply with **India's DPDP Act (2023)** → student consent, secure storage, access logs.
- **Encryption (SSL), audit trails, approval workflows** for profile edits.
- Protect sensitive data (salary, personal info).

## 8. User Experience & Accessibility

- Simple, intuitive, mobile-friendly interface.
- Personalized dashboard (highlight eligible companies, upcoming interviews).
- Multi-device compatibility (desktop, mobile).
- Easy navigation with search, filters, and graphical insights.

## 9. Placement Cell Analytics & Insights

- Track **student participation** in training/mocks.
- Resource usage analytics (which materials are effective).
- Company-wise and student-wise placement statistics.
- Feedback module for students post-interview/placement.
- Support **trend analysis** using industry reports (NASSCOM, LinkedIn).

## 10. System Technology & Scalability

- Built on a **reliable, scalable RDBMS** (PostgreSQL recommended).
  - Must ensure **ACID compliance** for data integrity.
  - Open-source and cost-effective for long-term sustainability.
  - Cross-platform with support for future extensions.
  - Capable of handling **large volumes of data** (students, recruiters, years of history).
-

## **User Classes and Characteristics**

### **1. Students**

- Role: Primary beneficiaries of the placement database.
- Responsibilities:
  - Create and maintain their placement profiles (academic details, skills, projects, certifications).
  - View company details, job roles, eligibility criteria, and placement statistics.
  - Apply to companies through the system (if feature enabled).
- Access Rights: Limited; can edit only their own profile (subject to Placement Cell approval).

### **2. Placement Cell Members (Administrators)**

- Role: Core managers and coordinators of the placement process.
- Responsibilities:
  - Manage all student records and verify/update profiles.
  - Upload company information, job details, and recruitment notices.
  - Generate placement reports (year-wise, branch-wise, company-wise).
  - Maintain official records and office letters (for NAAC/NIRF).
- Access Rights: Full access; can create, update, and approve records.

### **3. Faculty / Department Coordinators**

- Role: Academic advisors and coordinators supporting placements.
- Responsibilities:
  - Monitor placement readiness of students in their department.
  - Access student placement data and progress reports.
  - Provide recommendations or feedback on student preparedness.
- Access Rights: Restricted; read-only access to departmental student data and reports.

#### **4. Companies / Recruiters**

- Role: External stakeholders who hire students.
- Responsibilities:
  - View standardized student profiles and resumes.
  - Access institutional placement statistics and historical hiring trends.
  - Share job descriptions, eligibility criteria, and selection process details.
- Access Rights: Limited; can access only recruiter dashboard and relevant student data.

#### **5. Alumni (Optional User Class)**

- Role: Past students who may contribute to placements.
  - Responsibilities:
    - Share placement feedback, referrals, and opportunities.
    - Provide mentorship and guidance to current students.
  - Access Rights: Limited; can access placement trends and contribute feedback.
- 

### **Operating Environment**

#### **i. Hardware, Software, or Connectivity Requirements**

##### **Hardware Requirements**

- **Server-side (if hosted on institutional servers):**
  - Processor: Minimum Intel i5 or equivalent
  - RAM: 8 GB or higher
  - Storage: 500 GB or more (expandable for historical data & office letters)
  - Backup system or cloud storage support

- **Client-side (Students, Faculty, Companies):**
  - Any device with internet capability (Laptop/Desktop/Mobile/Tablet)
  - Minimum 2 GB RAM for smooth browser-based access

### **Software Requirements**

- **Server-side:**
  - Operating System: Linux/Windows Server
  - Database: MySQL / PostgreSQL / MongoDB
  - Backend Framework: Java / Python (Django/Flask) / PHP / Node.js
  - Frontend: HTML5, CSS3, JavaScript (React/Angular/Vue)
- **Client-side:**
  - Standard web browser (Chrome, Firefox, Edge, Safari)
  - No additional software installation required

### **Connectivity Requirements**

- High-speed internet connection (minimum 10 Mbps for server hosting; 2 Mbps for end-users)
- Secure LAN/Wi-Fi connectivity within the campus
- VPN or SSL encryption for secure remote access

## **ii. External Interface Requirements**

### **User Interfaces**

- Web-based portal (responsive for desktop and mobile)
- Mobile application (optional) for ease of access
- Graphical dashboards and visual reports

### **Hardware Interfaces**

- Compatible with standard PCs, laptops, and mobile devices

- Printer support for downloading and printing placement reports

### **Software Interfaces**

- Integration with existing institutional ERP or Student Information System (if available)
- APIs for data exchange with accreditation/ranking bodies (NAAC/NIRF reporting)
- Export/import options in common formats (CSV, Excel, PDF)

### **Communication Interfaces**

- Email and SMS notification system for alerts and updates
- Possible integration with WhatsApp/Telegram bots for placement notifications

### **iii. Hardware/Software/Third-Party APIs & External Sources**

- **Database Systems:** MySQL/PostgreSQL for structured data storage
  - **Visualization Tools:** Chart.js / D3.js / Recharts (for graphs & dashboards)
  - **Authentication:** Google/Facebook OAuth or institutional login (SSO)
  - **Cloud Services** (optional):
    - AWS/Azure/Google Cloud for hosting and backups
    - Firebase for mobile app backend support
  - **Third-Party APIs** (optional, if extended):
    - Email APIs (SendGrid, Gmail API) for notifications
    - SMS APIs (Twilio, MSG91) for placement alerts
    - Document storage APIs (Google Drive/OneDrive) for uploading office letters
  - **External Accreditation References:**
    - NAAC (for placement evidence)
    - NIRF (for placement metrics and ranking submissions)
-

## **Product Functions**

The Placement Database will provide the following key functions:

### **1. Student Module**

- Create and update personal placement profiles (academic records, skills, certifications, projects).
- View company details, job roles, eligibility criteria, and salary information.
- Access historical placement statistics (year-wise, branch-wise, company-wise).
- Receive notifications and alerts for placement drives, deadlines, and updates.
- Download or export personalized reports (resume, profile summary).

### **2. Placement Cell Module**

- Add, update, and verify student records.
- Upload company details, recruitment processes, and eligibility criteria.
- Approve student profile updates before final submission.
- Generate automated reports (branch-wise, company-wise, salary/package statistics).
- Maintain and manage official office letters (offer letters, MoUs) for NAAC/NIRF evidence.
- Control role-based access for all system users.

### **3. Faculty/Department Module**

- View placement progress of students within their department.
- Access departmental placement statistics and reports.
- Provide academic or skill-related recommendations (optional feature).

### **4. Company/Recruiter Module**

- Submit job role details, eligibility requirements, and hiring processes.
- View standardized student profiles for shortlisting.
- Access institutional placement statistics and historical trends.
- Download candidate lists or reports in secure formats.

## **5. Alumni Module**

- Share placement experiences, referrals, or opportunities.
- Provide mentorship and feedback for current students.
- Access general placement statistics and trends.

## **6. System-Level Functions**

- Provide secure login and authentication for all users.
  - Role-based access control to ensure data confidentiality.
  - Advanced search and filter options for quick data retrieval.
  - Data export/import support (CSV, Excel, PDF).
  - Graphical dashboards for quick insights and decision-making.
  - Backup and recovery functions to ensure data security.
-



## **Privileges**

The Placement Database system will implement a role-based access control (RBAC) system to ensure that each user class has access only to the modules and functions relevant to their role. This structured approach protects sensitive data and prevents unauthorized access to critical functions. Each user class will have a defined set of privileges, ensuring a secure and efficient workflow.

### **A. Student Module**

Students are the primary beneficiaries and end-users of the system. Their privileges are limited to viewing and managing their personal data.

- **Create and Maintain Profile:** Students can create and update their personal placement profiles, including academic details, skills, projects, and certifications. All profile edits require approval from the Placement Cell before final submission.
- **View Information:** Students can view company details, job roles, eligibility criteria, and salary information. They can also access historical placement statistics, such as year-wise, branch-wise, and company-wise data.
- **Apply to Companies:** Students can apply to companies through the system if the feature is enabled.
- **Access Notifications:** Students can receive notifications and alerts for upcoming deadlines, recruitment events, or profile updates.
- **Download Reports:** They have the privilege to download or export personalized reports, such as a resume or a profile summary.

### **B. Placement Cell Module (Administrators)**

Placement Cell members serve as the core managers and coordinators of the system with full access privileges.

- **Data Management:** They can add, update, and verify all student records. They also have the privilege to upload company information, job details, and recruitment notices.
- **Approval & Control:** Placement Cell members are responsible for approving student profile updates and managing official records and letters, which are required for institutional reporting and accreditation.
- **Reporting:** They can generate automated placement reports based on various criteria, such as branch, company, and salary statistics. They can also control role-based access for all system users.

### **C. Faculty / Department Coordinators Module**

Faculty members have restricted, read-only access to the system, primarily for monitoring and verification purposes.

- **View Reports:** They can view placement progress and student data for their specific department.
- **Download Data:** They have the privilege to download candidate lists or reports in a secure format.
- **Mentorship & Feedback:** They can access student data and progress reports to provide recommendations or feedback on student preparedness.

#### **D. Companies / Recruiters Module**

Recruiters are external stakeholders with limited access to standardized student profiles and reports.

- **View Profiles:** They can view standardized student profiles and resumes.
- **Access Statistics:** Recruiters can access institutional placement statistics and historical hiring trends to benchmark their recruitment outcomes.
- **Manage Job Postings:** They have the privilege to share job descriptions, eligibility criteria, and selection process details.

#### **E. Alumni Module (Optional)**

Alumni have limited access to placement trends and can contribute feedback and referrals.

- **View Trends:** They can access general placement statistics and trends to stay informed.
  - **Contribution:** They have the privilege to share placement feedback, referrals, and new job opportunities. They can also provide mentorship and guidance to current students.
-

## **Assumptions**

The development of the Placement Database is predicated on a set of precise assumptions that reflect real-world university and corporate environments. These assumptions are crucial for defining the scope of the project and ensuring its feasibility and success.

### **A. Operational and User Assumptions**

- **User Adoption and Engagement:** It is assumed that all identified user classes—students, Placement Cell members, faculty, and recruiters—will actively engage with and adopt the new system. This includes an assumption that students will consistently create and maintain their profiles with accurate information, as this is a common issue in many institutions. The system's success relies on a shift from a manual, scattered approach to a centralized, digital one.
- **Data Quality and Integrity:** A key assumption is that the Placement Cell will have the necessary resources and dedicated personnel to perform the ongoing verification and approval of student profile updates, a critical step to prevent incorrect data from entering the system. Furthermore, it is assumed that the initial data migrated from existing platforms like spreadsheets and emails will be cleansed and validated to ensure accuracy.
- **Organizational Support:** It is assumed that the university administration will provide full support for the project, including the allocation of time and resources for comprehensive training and onboarding sessions for all stakeholders. This is essential to ensure proper usage and to justify the system's curriculum reforms and marketing to prospective students.
- **Communication Channels:** The assumption is that all stakeholders will be able to access the system's communication interfaces, such as email and SMS notifications, to stay informed about deadlines and events. We also assume that external communication channels like email will remain a primary mode of communication for official documents like offer letters.

### **B. Technical and System Assumptions**

- **Scalability and Performance:** The system assumes that the selected hardware and software stack (e.g., MySQL, Intel i5 processor, 8 GB RAM) is sufficient to handle expected user traffic, especially during high-peak events like mass registrations or interview scheduling. It is assumed that the database can scale to accommodate growing numbers of student records and historical data without a significant loss in performance.
- **Integration with External Systems:** The assumption is that APIs or other integration methods with the existing university ERP or Student Information

- Systems are available and properly documented. This is crucial for seamless data exchange and avoiding manual, redundant data entry.
  - **Robust Backup and Recovery:** We assume that a robust, automated backup system and a disaster recovery process will be implemented as part of the solution. This is a critical assumption to mitigate the real-world risk of data loss due to crashes, which is a known problem in existing systems.
  - **Standardized Data Formats:** We assume that all incoming data, whether from bulk uploads (e.g., Excel/CSV) or API integrations, will conform to a standardized format that the database can easily process and store. This simplifies the data ingestion process and reduces the risk of data inconsistencies.
-

## **Business Constraints**

Business constraints are the limitations that a project must operate within. For the Placement Database, these constraints are primarily related to organizational policies, legal requirements, and existing infrastructure.

### **A. Organizational Constraints**

- **Integration with Existing Systems:** The new database may need to integrate with existing institutional systems, such as an ERP or Student Information System. This integration must be seamless and not disrupt current operations.
- **Data Ownership and Policies:** The university retains full ownership of all data stored in the database. The system must comply with institutional policies regarding data retention, privacy, and security.
- **Approval & Verification Workflow:** The system must strictly adhere to the university's official workflow, such as the requirement for Placement Cell approval on all student profile edits.

### **B. Legal and Ethical Constraints**

- **Data Privacy:** The system must comply with data privacy regulations to protect sensitive personal information, such as student contact details and salary packages. Role-based access control is a crucial component for meeting this requirement.
- **NAAC/NIRF Reporting:** The database must be capable of generating specific reports and storing official documents (e.g., offer letters, MoUs) that are required for institutional accreditation and ranking processes like NAAC and NIRF.
- **Ethical Data Usage:** The data collected must only be used for its stated purpose—facilitating the placement process. It should not be used for unrelated activities like marketing to third parties without explicit consent.

### **C. Technical and Environmental Constraints**

- **Technology Stack:** The project is constrained to a specific technology stack as outlined in the Operating Environment section, including the use of MySQL/PostgreSQL/MongoDB for the database and frameworks like Java/Python/Node.js for the backend.
- **Backup and Recovery:** The system must implement a robust backup and recovery plan to prevent data loss in the event of a crash or server malfunction. The plan should ensure that incomplete transactions are finished and that data integrity is maintained.

- **Scalability:** The system must be designed to handle a growing number of students, companies, and historical data over time. The database structure must be scalable to accommodate future growth without significant performance degradation.
- 

## **Conclusion**

The SRS delivers a centralized, secure, and future-ready Placement Management System, replacing outdated manual methods and creating a transparent, data-driven placement ecosystem.

## **Core Focus Areas:**

- **Centralized Data Hub** – Student profiles, company details, placement drives, interviews, and offers all in one place.
- **Intelligent Scheduling** – Calendar view, automated interview slot booking, conflict detection, and real-time updates.
- **Resource & Preparation Hub** – Aptitude/coding practice, company profiles, past year trends, and curated prep material.
- **Actionable Analytics** – Recruiter participation trends, salary insights, student progress tracking, NAAC/NIRF-ready reports.

## **Data, Security & Compliance:**

- **Normalized, Scalable Database** – Eliminates duplication, improves speed, and handles growing student volumes.
- **Role-Based Access Control (RBAC)** – Granular permissions for students, placement cell, faculty, and recruiters.
- **Data Protection & DPDP Act 2023 Compliance** – SSL encryption, audit logs, and approval workflows.
- **Crash Recovery & Backup Strategy** – Automated backups, transaction integrity, and zero data loss even under failures.

**User Roles & Personalization:**

- **Students** – Profiles, eligible jobs, interview schedules, and performance tracking.
- **Placement Cell** – Full control over data, schedules, reports, and document management.
- **Faculty Coordinators** – Read-only access to guide students and monitor progress.
- **Recruiters** – Secure access to student profiles, statistics, and candidate lists.
- **Alumni** – Mentorship, referrals, and trend analysis.

**System & Operating Environment:**

- **Tech Stack:** PostgreSQL/MySQL, secure cloud/hosted servers, web + mobile access.
- **Performance Ready:** Designed for peak loads during mass registrations and drives.
- **Scalable:** Supports multiple academic years, historical data, and future ERP integrations.

**Assumptions & Constraints:**

- Verified student/company master data provided by Placement Cell.
- Reliable internet assumed; fallback notification and offline data capture supported.
- Budget/time constraints handled via phased rollout of features.

**Key Differentiators:**

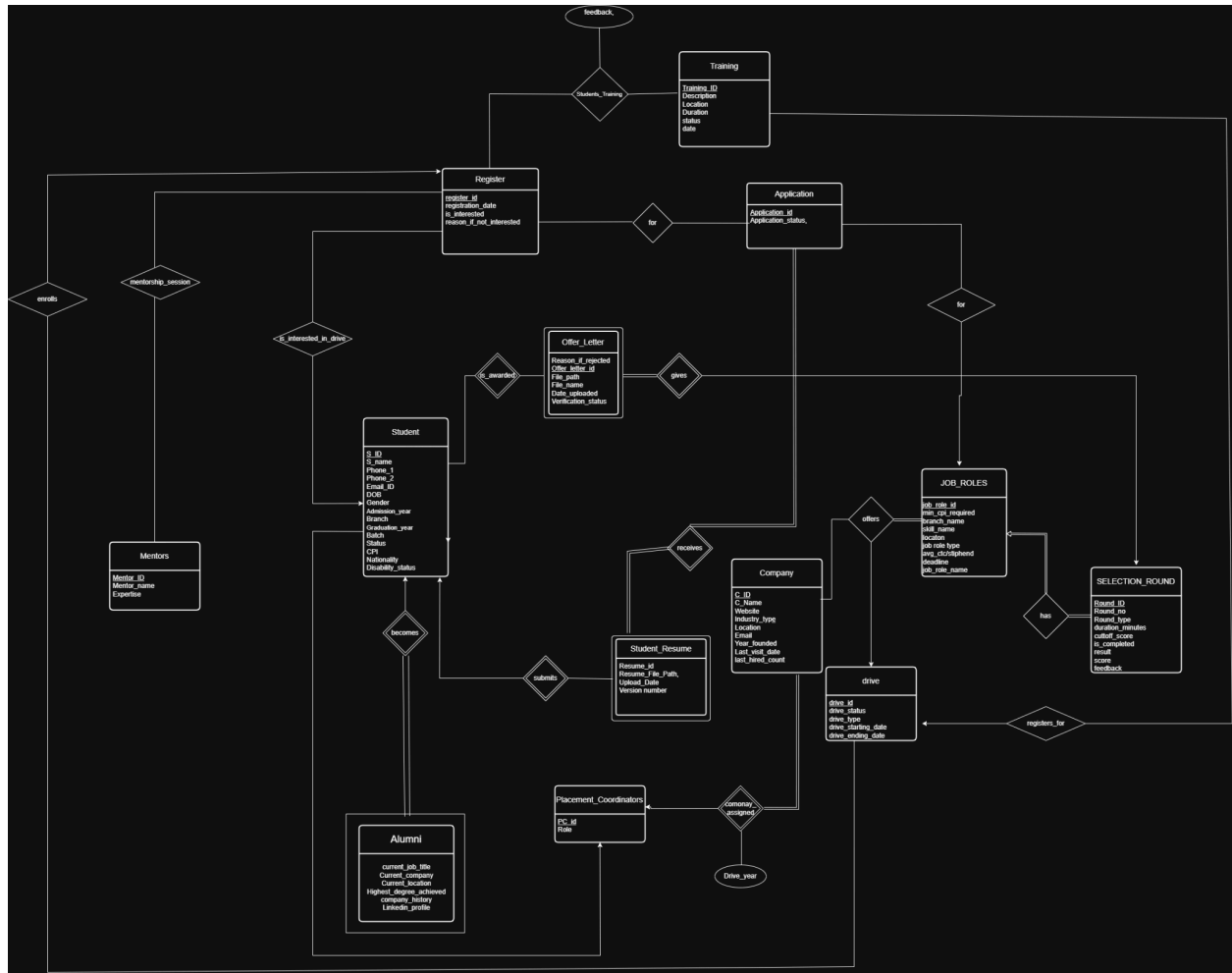
- **Transparency:** All stakeholders see real-time, accurate data.
- **Efficiency:** Reduces manual work and delays by >70%.
- **Insights:** Enables data-driven decisions for training, recruiter targeting, and curriculum improvement.
- **Credibility:** Strengthens institutional reporting and improves NAAC/NIRF ranking readiness.

**Success Criteria:**

- Faster communication, fewer missed deadlines, and smooth drive management.
- Accurate, exportable reports (PDF/Excel) for all stakeholders.
- Higher student participation, satisfaction, and placement percentages.
- Recruiters experience faster shortlisting and improved candidate quality.

## ER DIAGRAM

[https://drive.google.com/file/d/1yk1\\_iZtKsKbquvMNg-e2mbpIvATtpFff/view?usp=sharing](https://drive.google.com/file/d/1yk1_iZtKsKbquvMNg-e2mbpIvATtpFff/view?usp=sharing)





## **2. Relational Tables with All Constraints**

### **2.1 STUDENT**

STUDENT(S\_ID, S\_Name, Email\_ID, DOB, Gender, Admission\_Year, Branch, Graduation\_Year, Status, CPI, Nationality, Disability\_Status)

Primary Key: S\_ID

Unique: Email\_ID

Not Null: S\_Name, Email\_ID, DOB, Admission\_Year, Branch, Status, Nationality

Check Constraints:

- Gender IN ('Male', 'Female', 'Other')
- Status IN ('Active', 'Graduated', 'Dropped')
- Nationality IN ('Indian', 'Foreign')
- Disability\_Status IN ('Yes', 'None')
- CPI BETWEEN 0.00 AND 10.00

### **2.2 STUDENT\_PHONE**

STUDENT\_PHONE(S\_ID, Phone\_No1, Phone\_No2)

Primary Key: S\_ID

Foreign Key: S\_ID REFERENCES STUDENT(S\_ID) ON DELETE CASCADE

Not Null: S\_ID, Phone\_No1

### **2.3 ALUMNI**

ALUMNI(S\_ID, Current\_Job\_Title, Current\_Company, Current\_Location, Highest\_Degree\_Achieved, Company\_History, LinkedIn\_Profile)

Primary Key: S\_ID

Foreign Key: S\_ID REFERENCES STUDENT(S\_ID) ON DELETE CASCADE

Not Null: S\_ID, Current\_Job\_Title, Current\_Company

### **2.4 COMPANY**

COMPANY(C\_ID, C\_Name, Website, Industry\_Type, Location, Email, Year\_Founded, Last\_Visit\_Year, Last\_Hired\_Count)

Primary Key: C\_ID

Unique: Email

Not Null: C\_ID, C\_Name

## **2.5 DRIVE**

DRIVE(Drive\_ID, Drive\_Start\_Date, Drive\_End\_Date, Drive\_Status)

Primary Key: Drive\_ID

Not Null: Drive\_ID, Drive\_Start\_Date

Check Constraints:

- Drive\_Status IN ('Upcoming', 'Ongoing', 'Completed')
- Drive\_End\_Date >= Drive\_Start\_Date (domain constraint)

Default: Drive\_Status = 'Upcoming'

## **2.6 DRIVE\_LOG**

DRIVE\_LOG(Drive\_ID, Drive\_type)

Primary Key: Drive\_ID

Foreign Key: Drive\_ID REFERENCES DRIVE(Drive\_ID) ON DELETE CASCADE

Unique: Drive\_type

Not Null: Drive\_type

## **2.7 DRIVE\_ELIGIBILITY**

DRIVE\_ELIGIBILITY(Drive\_ID, Batch\_Eligible, Batch\_year\_Eligible)

Primary Key: (Drive\_ID, Batch\_Eligible)

Foreign Key: Drive\_ID REFERENCES DRIVE(Drive\_ID) ON DELETE CASCADE

Not Null: Drive\_ID, Batch\_Eligible, Batch\_year\_Eligible

## **2.8 JOB\_ROLE**

JOB\_ROLE(Job\_Role\_ID, C\_ID, Drive\_ID, Avg\_CTC\_or\_Stipend, Deadline)

Primary Key: Job\_Role\_ID

Foreign Keys:

- C\_ID REFERENCES COMPANY(C\_ID) ON DELETE CASCADE
- Drive\_ID REFERENCES DRIVE(Drive\_ID) ON DELETE CASCADE

Not Null: Job\_Role\_ID, C\_ID, Drive\_ID

## **2.9 JOB\_ROLE\_ACADEMIC\_ELIGIBILITY**

JOB\_ROLE\_ACADEMIC\_ELIGIBILITY(Job\_Role\_ID, Branch\_name, Min\_CPI\_Required)

Primary Key: (Job\_Role\_ID, Branch\_name)

Foreign Key: Job\_Role\_ID REFERENCES JOB\_ROLE(Job\_Role\_ID) ON DELETE CASCADE

Not Null: Job\_Role\_ID, Branch\_name, Min\_CPI\_Required

Check Constraints:

- Min\_CPI\_Required BETWEEN 0.00 AND 10.00

## **2.10 JOB\_ROLE\_SKILL\_REQUIREMENT**

JOB\_ROLE\_SKILL\_REQUIREMENT(Job\_Role\_ID, Skill\_Name)

Primary Key: (Job\_Role\_ID, Skill\_Name)

Foreign Key: Job\_Role\_ID REFERENCES JOB\_ROLE(Job\_Role\_ID) ON DELETE CASCADE

Not Null: Job\_Role\_ID, Skill\_Name

## **2.11 JOB\_ROLE\_LOCATION**

JOB\_ROLE\_LOCATION(Job\_Role\_ID, Location)

Primary Key: (Job\_Role\_ID, Location)

Foreign Key: Job\_Role\_ID REFERENCES JOB\_ROLE(Job\_Role\_ID) ON DELETE CASCADE

Not Null: Job\_Role\_ID, Location

## **2.12 REGISTER**

REGISTER(Register\_ID, S\_ID, Drive\_ID, Registration\_Date,  
Is\_Interested, Reason\_If\_Not\_Interested)

Primary Key: Register\_ID

Foreign Keys:

- S\_ID REFERENCES STUDENT(S\_ID) ON DELETE CASCADE

- Drive\_ID REFERENCES DRIVE(Drive\_ID) ON DELETE CASCADE

Unique: (S\_ID, Drive\_ID)

Default: Is\_Interested = TRUE

## **2.13 STUDENT\_RESUME**

STUDENT\_RESUME(Resume\_ID, S\_ID, Resume\_File\_Path, Upload\_Date, Version\_Number)

Primary Key: Resume\_ID,s\_id

Foreign Key: S\_ID REFERENCES STUDENT(S\_ID) ON DELETE CASCADE

Not Null: Resume\_ID, S\_ID, Resume\_File\_Path, Upload\_Date, Version\_Number

## **2.14 APPLICATION**

APPLICATION(Application\_ID, Register\_ID, Job\_Role\_ID, Resume\_ID,  
Application\_Status)

Primary Key: Application\_ID

Foreign Keys:

- Register\_ID REFERENCES REGISTER(Register\_ID) ON DELETE CASCADE
- Job\_Role\_ID REFERENCES JOB\_ROLE(Job\_Role\_ID) ON DELETE CASCADE
- Resume\_ID REFERENCES STUDENT\_RESUME(Resume\_ID)

Unique: (Register\_ID, Job\_Role\_ID)

Check Constraints:

- Application\_Status IN ('Applied', 'Shortlisted', 'Interviewed',  
'Selected', 'Rejected')

## **2.15 SELECTION\_ROUND**

SELECTION\_ROUND(Round\_ID, Job\_Role\_ID, Round\_Number, Round\_Type,  
Duration\_Minutes, Cutoff\_Score, Is\_Completed)

Primary Key: Round\_ID

Foreign Key: Job\_Role\_ID REFERENCES JOB\_ROLE(Job\_Role\_ID) ON DELETE  
CASCADE

Unique: (Job\_Role\_ID, Round\_Number)

Not Null: Round\_ID, Job\_Role\_ID, Round\_Number, Round\_Type

Check Constraints:

- Round\_Type IN ('Aptitude Test', 'Technical Test', 'Technical Interview',  
'HR Interview', 'Case Study', 'Coding Test')

Default: Is\_Completed = TRUE

## **2.16 STUDENT\_ROUND\_RESULT**

STUDENT\_ROUND\_RESULT(Result\_ID, Round\_ID, Application\_ID, Result,  
Score, Feedback)

Primary Key: Result\_ID

Foreign Keys:

- Round\_ID REFERENCES SELECTION\_ROUND(Round\_ID) ON DELETE CASCADE
- Application\_ID REFERENCES APPLICATION(Application\_ID) ON DELETE CASCADE

Unique: (Round\_ID, Application\_ID)

Check Constraints:

- Result IN ('Pending', 'Passed', 'Failed')

## **2.17 OFFER\_LETTER**

OFFER\_LETTER(Offer\_Letter\_ID, S\_ID, Job\_Role\_ID, File\_Path, File\_Name,  
Date\_Uploaded, Verification\_Status, Reason\_if\_rejected)

Primary Key: Offer\_Letter\_ID

Foreign Keys:

- S\_ID REFERENCES STUDENT(S\_ID) ON DELETE CASCADE
- Job\_Role\_ID REFERENCES JOB\_ROLE(Job\_Role\_ID) ON DELETE CASCADE

Unique: (S\_ID, Job\_Role\_ID)

Not Null: Offer\_Letter\_ID, S\_ID, Job\_Role\_ID

Check Constraints:

- Verification\_Status IN ('Pending', 'Verified', 'Rejected')

Default: Verification\_Status = 'Pending', Date\_Uploaded = CURRENT\_TIMESTAMP

## **2.18 PLACEMENT\_COORDINATORS**

PLACEMENT\_COORDINATORS(PC\_ID, S\_ID, Role)

Primary Key: PC\_ID

Foreign Key: S\_ID REFERENCES STUDENT(S\_ID) ON DELETE CASCADE

Not Null: PC\_ID, S\_ID

## **2.19 COMPANY\_ASSIGNED**

COMPANY\_ASSIGNED(C\_ID, PC\_ID)

Primary Key: (C\_ID, PC\_ID)

Foreign Keys:

- C\_ID REFERENCES COMPANY(C\_ID) ON DELETE CASCADE
- PC\_ID REFERENCES PLACEMENT\_COORDINATORS(PC\_ID) ON DELETE CASCADE

Not Null: C\_ID, PC\_ID

## **2.20 MENTORS**

MENTORS(Mentor\_ID, Mentor\_Name, Expertise)

Primary Key: Mentor\_ID

Not Null: Mentor\_ID, Mentor\_Name

## **2.21 MENTOR\_STUDENT**

MENTOR\_STUDENT(Mentor\_ID, S\_ID)

Primary Key: (Mentor\_ID, S\_ID)

Foreign Keys:

- Mentor\_ID REFERENCES MENTORS(Mentor\_ID) ON DELETE CASCADE
- S\_ID REFERENCES STUDENT(S\_ID) ON DELETE CASCADE

Not Null: Mentor\_ID, S\_ID

## **2.22 TRAINING**

TRAINING(Training\_ID, Description, Location, Duration, Date, Status)

Primary Key: Training\_ID

Not Null: Training\_ID

Check Constraints:

- Status IN ('Completed', 'Upcoming')

## **2.23 STUDENT\_TRAINING**

STUDENT\_TRAINING(S\_ID, Training\_ID, Feedback)

Primary Key: (S\_ID, Training\_ID)

Foreign Keys:

- S\_ID REFERENCES STUDENT(S\_ID) ON DELETE CASCADE
- Training\_ID REFERENCES TRAINING(Training\_ID) ON DELETE CASCADE

Not Null: S\_ID, Training\_ID

## Normalized Tables

### Table 1: STUDENT

Attributes: S\_ID, S\_Name, Email\_ID, DOB, Gender, Admission\_Year, Branch, Graduation\_Year, Status, CPI, Nationality, Disability\_Status

Primary Key: S\_ID

Candidate Keys: S\_ID, Email\_ID

Normal Form: BCNF

Reason: All attributes fully dependent on S\_ID only, no transitive dependencies

### Table 2: COMPANY

Attributes: C\_ID, C\_Name, Website, Industry\_Type, Location, Email, Year\_Founded, Last\_Visit\_Year, Last\_Hired\_Count

Primary Key: C\_ID

Candidate Keys: C\_ID, Email

Normal Form: BCNF

Reason: All attributes fully dependent on C\_ID only

### Table 3: DRIVE

Attributes: Drive\_ID, Drive\_Start\_Date, Drive\_End\_Date, Drive\_Status

Primary Key: Drive\_ID

Normal Form: BCNF

Reason: Single key attribute, all others fully dependent on Drive\_ID

### Table 4: JOB\_ROLE

Attributes: Job\_Role\_ID, C\_ID, Drive\_ID, Avg\_CTC\_or\_Stipend, Deadline

Primary Key: Job\_Role\_ID

Foreign Keys: C\_ID → COMPANY, Drive\_ID → DRIVE

Normal Form: BCNF

Reason: All attributes dependent on Job\_Role\_ID, FKs reference PKs

### Table 5: MENTORS

Attributes: Mentor\_ID, Mentor\_Name, Expertise

Primary Key: Mentor\_ID

Normal Form: BCNF

Reason: Simple entity with direct dependencies

**Table 6: TRAINING**

Attributes: Training\_ID, Description, Location, Duration, Date, Status

Primary Key: Training\_ID

Normal Form: BCNF

Reason: All attributes dependent on Training\_ID only

**4.2 Dependent Entity Tables (Weak Entities)**

**Table 7: STUDENT\_PHONE**

Attributes: S\_ID, Phone\_No1, Phone\_No2

Primary Key: S\_ID

Foreign Key: S\_ID → STUDENT

Normal Form: BCNF

Reason: Existence-dependent on STUDENT, no repeating groups

**Table 8: ALUMNI**

Attributes: S\_ID, Current\_Job\_Title, Current\_Company, Current\_Location,  
Highest\_Degree\_Achieved, Company\_History, LinkedIn\_Profile

Primary Key: S\_ID

Foreign Key: S\_ID → STUDENT

Normal Form: BCNF

Reason: Alumni information specific to graduated students

**Table 9: PLACEMENT\_COORDINATORS**

Attributes: PC\_ID, S\_ID, Role

Primary Key: PC\_ID

Foreign Key: S\_ID → STUDENT

Normal Form: BCNF

Reason: Coordinator role dependent on PC\_ID

**Table 10: DRIVE\_LOG**

Attributes: Drive\_ID, Drive\_type

Primary Key: Drive\_ID



Foreign Key: Drive\_ID → DRIVE

Normal Form: BCNF

Reason: Additional drive metadata, 1:1 relationship

#### **Table 11: DRIVE\_ELIGIBILITY**

Attributes: Drive\_ID, Batch\_Eligible, Batch\_year\_Eligible

Primary Key: (Drive\_ID, Batch\_Eligible)

Foreign Key: Drive\_ID → DRIVE

Normal Form: BCNF

Reason: Multi-valued attribute (eligible batches) properly decomposed

### **4.3 Relationship Tables (Many-to-Many Resolution)**

#### **Table 12: REGISTER (Student-Drive Registration)**

Attributes: Register\_ID, S\_ID, Drive\_ID, Registration\_Date,  
Is\_Interested, Reason\_If\_Not\_Interested

Primary Key: Register\_ID

Candidate Key: (S\_ID, Drive\_ID)

Foreign Keys: S\_ID → STUDENT, Drive\_ID → DRIVE

Normal Form: BCNF

Reason: Resolves M:N between STUDENT and DRIVE with registration attributes

#### **Table 13: APPLICATION (Student Job Application)**

Attributes: Application\_ID, Register\_ID, Job\_Role\_ID, Resume\_ID,  
Application\_Status

Primary Key: Application\_ID

Candidate Key: (Register\_ID, Job\_Role\_ID)

Foreign Keys: Register\_ID → REGISTER, Job\_Role\_ID → JOB\_ROLE,  
Resume\_ID → STUDENT\_RESUME

Normal Form: BCNF

Reason: Links registration to specific job roles, includes application metadata

#### **Table 14: STUDENT\_RESUME**

Attributes: Resume\_ID, S\_ID, Resume\_File\_Path, Upload\_Date, Version\_Number

Primary Key: Resume\_ID, S\_ID

Foreign Key: S\_ID → STUDENT

Normal Form: BCNF

Reason: Students can have multiple resume versions

**Table 15: SELECTION\_ROUND**

Attributes: Round\_ID, Job\_Role\_ID, Round\_Number, Round\_Type,  
Duration\_Minutes, Cutoff\_Score, Is\_Completed

Primary Key: Round\_ID

Candidate Key: (Job\_Role\_ID, Round\_Number)

Foreign Key: Job\_Role\_ID → JOB\_ROLE

Normal Form: BCNF

Reason: Multiple rounds per job role with ordering

**Table 16: STUDENT\_ROUND\_RESULT**

Attributes: Result\_ID, Round\_ID, Application\_ID, Result, Score, Feedback

Primary Key: Result\_ID

Candidate Key: (Round\_ID, Application\_ID)

Foreign Keys: Round\_ID → SELECTION\_ROUND, Application\_ID → APPLICATION

Normal Form: BCNF

Reason: Resolves M:N between APPLICATION and SELECTION\_ROUND with results

**Table 17: OFFER\_LETTER**

Attributes: Offer\_Letter\_ID, S\_ID, Job\_Role\_ID, File\_Path, File\_Name,  
Date\_Uploaded, Verification\_Status, Reason\_if\_rejected

Primary Key: Offer\_Letter\_ID

Candidate Key: (S\_ID, Job\_Role\_ID)

Foreign Keys: S\_ID → STUDENT, Job\_Role\_ID → JOB\_ROLE

Normal Form: BCNF

Reason: Documents final placement offers with verification workflow

**Table 18: COMPANY\_ASSIGNED**

Attributes: C\_ID, PC\_ID

Primary Key: (C\_ID, PC\_ID)

Foreign Keys: C\_ID → COMPANY, PC\_ID → PLACEMENT\_COORDINATORS

Normal Form: BCNF

Reason: Pure M:N resolution between coordinators and companies

**Table 19: MENTOR\_STUDENT**

Attributes: Mentor\_ID, S\_ID

Primary Key: (Mentor\_ID, S\_ID)

Foreign Keys: Mentor\_ID → MENTORS, S\_ID → STUDENT

Normal Form: BCNF

Reason: Pure M:N resolution between mentors and students

**Table 20: STUDENT\_TRAINING**

Attributes: S\_ID, Training\_ID, Feedback

Primary Key: (S\_ID, Training\_ID)

Foreign Keys: S\_ID → STUDENT, Training\_ID → TRAINING

Normal Form: BCNF

Reason: Resolves M:N between students and training with feedback

**4.4 Multi-valued Attribute Tables****Table 21: JOB\_ROLE\_ACADEMIC\_ELIGIBILITY**

Attributes: Job\_Role\_ID, Branch\_name, Min\_CPI\_Required

Primary Key: (Job\_Role\_ID, Branch\_name)

Foreign Key: Job\_Role\_ID → JOB\_ROLE

Normal Form: BCNF

Reason: Academic criteria varies by branch for each role

**Table 22: JOB\_ROLE\_SKILL\_REQUIREMENT**

Attributes: Job\_Role\_ID, Skill\_Name

Primary Key: (Job\_Role\_ID, Skill\_Name)

Foreign Key: Job\_Role\_ID → JOB\_ROLE

Normal Form: BCNF

Reason: Multi-valued attribute (required skills) properly normalized

**Table 23: JOB\_ROLE\_LOCATION**

Attributes: Job\_Role\_ID, Location

Primary Key: (Job\_Role\_ID, Location)

Foreign Key: Job\_Role\_ID → JOB\_ROLE

Normal Form: BCNF

Reason: Multi-valued attribute (job locations) properly normalized

## DDL

```
CREATE TABLE Student (  
    S_ID VARCHAR(10) PRIMARY KEY,  
    S_Name VARCHAR(100) NOT NULL,  
    Email_ID VARCHAR(100) UNIQUE NOT NULL,  
    DOB DATE NOT NULL,  
    Gender VARCHAR(10) CHECK (Gender IN ('Male', 'Female', 'Other')),  
    Admission_Year INT NOT NULL,  
    Branch VARCHAR(50) NOT NULL,  
    Graduation_Year INT,  
    Status VARCHAR(20) CHECK (Status IN ('Active', 'Graduated', 'Dropped')) NOT NULL,  
    CPI NUMERIC(3, 2),  
    Nationality VARCHAR(50) CHECK (Nationality IN ('Indian', 'Foreign')) NOT NULL,  
    Disability_Status VARCHAR(10) CHECK (Disability_Status IN ('Yes', 'None'))  
);
```

```
CREATE TABLE Student_Phone (  
    S_ID VARCHAR(10) NOT NULL,  
    Phone_No1 VARCHAR(15) NOT NULL,  
    Phone_No2 VARCHAR(15),  
    PRIMARY KEY (S_ID),  
    FOREIGN KEY (S_ID) REFERENCES Student(S_ID)  
);
```

```
CREATE TABLE registers (  
    Register_ID SERIAL PRIMARY KEY,  
    S_ID VARCHAR(10) REFERENCES Student(S_ID) ON DELETE CASCADE,  
    Drive_ID VARCHAR(10) REFERENCES Drive(Drive_ID) ON DELETE CASCADE,  
    Registration_Date DATE,  
    Is_Interested BOOLEAN DEFAULT TRUE,  
    Reason_If_Not_Interested TEXT,  
    CONSTRAINT registers_unique UNIQUE (S_ID, Drive_ID)  
);
```

```
CREATE TABLE Application (  
    Application_ID SERIAL PRIMARY KEY,
```

```

Register_ID INT REFERENCES registers(Register_ID) ON DELETE CASCADE,
Job_Role_ID VARCHAR(10) REFERENCES Job_Role(Job_Role_ID) ON DELETE
CASCADE,
Resume_ID VARCHAR(10) REFERENCES Student_Resume(Resume_ID),
Application_Status VARCHAR(20) CHECK (Application_Status IN
('Applied', 'Shortlisted', 'Interviewed', 'Selected', 'Rejected')
),

```

```

CREATE TABLE Alumni (
S_ID VARCHAR(10) PRIMARY KEY,
Current_Job_Title VARCHAR(100) NOT NULL,
Current_Company VARCHAR(100) NOT NULL,
Current_Location VARCHAR(100),
Highest_Degree_Achieved VARCHAR(50),
Company_History TEXT,
LinkedIn_Profile VARCHAR(255),
CONSTRAINT fk_alumni_student
FOREIGN KEY (S_ID)
REFERENCES Student(S_ID)
-- ON DELETE CASCADE ensures that if a student record is deleted,
-- the corresponding alumni record is also deleted.
ON DELETE CASCADE
);

```

```

CREATE TABLE Company (
C_ID VARCHAR(10) PRIMARY KEY,
C_Name VARCHAR(255) NOT NULL,
Website VARCHAR(255),
Industry_Type VARCHAR(100),
Location VARCHAR(255),
Email VARCHAR(100) UNIQUE,
Year_Founded YEAR,
Last_Visit_Year INT, -- Changed from Last_Visit_Date
Last_Hired_Count INT
);

```

```

CREATE TABLE Drive (
Drive_ID SERIAL PRIMARY KEY,

```

```
Drive_Start_Date DATE NOT NULL,  
Drive_End_Date DATE,  
Drive_Status VARCHAR(20) DEFAULT 'Upcoming'  
    CHECK (Drive_Status IN ('Upcoming', 'Ongoing', 'Completed')),  
);
```

```
CREATE TABLE Drive_log (  
    Drive_ID VARCHAR(10) NOT NULL,  
    Drive_type VARCHAR(100) UNIQUE NOT NULL,
```

```
    FOREIGN KEY (Drive_ID) REFERENCES Drive(Drive_ID) ON DELETE CASCADE  
);
```

```
CREATE TABLE Drive_Eligibility (  
    Drive_ID VARCHAR(10) NOT NULL REFERENCES Drive(Drive_ID) ON DELETE  
CASCADE ,  
    Batch_Eligible VARCHAR(20) NOT NULL,  
    Batch_year_Eligible VARCHAR(4) NOT NULL,  
    PRIMARY KEY (Drive_ID, Batch_Eligible)  
);
```

```
CREATE TABLE Job_Role (  
    Job_Role_ID VARCHAR(10) PRIMARY KEY,  
    C_ID VARCHAR(10) NOT NULL REFERENCES Company(C_ID) ON DELETE  
CASCADE,  
    Drive_ID VARCHAR(10) NOT NULL REFERENCES Drive(Drive_ID) ON DELETE  
CASCADE,  
    Avg_CTC_or_Stipend NUMERIC(10,2),  
    Deadline DATE,  
);
```

```

CREATE TABLE Job_Role_Academic_Eligibility (
    Job_Role_ID VARCHAR(10) NOT NULL REFERENCES Job_Role(Job_Role_ID) ON
DELETE CASCADE,
    Branch_name VARCHAR(15) NOT NULL,
    Min_CPI_Required NUMERIC(3, 2) NOT NULL,
    PRIMARY KEY (Job_Role_ID, Branch_name)
);

```

```

CREATE TABLE Job_Role_Skill_Requirement (
    Job_Role_ID VARCHAR(10) NOT NULL REFERENCES Job_Role(Job_Role_ID) ON
DELETE CASCADE,
    Skill_Name VARCHAR(50) NOT NULL,
    PRIMARY KEY (Job_Role_ID, Skill_Name)
);

```

```

CREATE TABLE Job_Role_Location (
    Job_Role_ID VARCHAR(10) REFERENCES Job_Role(Job_Role_ID) ON DELETE
CASCADE,
    Location VARCHAR(100) NOT NULL,
    PRIMARY KEY (Job_Role_ID, Location)
);

```

```

    UNIQUE (Register_ID, Job_Role_ID)
);

```

```

CREATE TABLE Selection_Round (
    Round_ID SERIAL PRIMARY KEY,
    Job_Role_ID VARCHAR(10) REFERENCES Job_Role(Job_Role_ID) ON DELETE
CASCADE,
    Round_Number INT NOT NULL, -- 1, 2, 3, etc.
    Round_Type VARCHAR(50) NOT NULL CHECK (Round_Type IN
('Aptitude Test', 'Technical Test',
'Technical Interview', 'HR Interview', 'Case Study',
'Coding Test')
),
    Duration_Minutes INT,
    Cutoff_Score DECIMAL(5,2), -- Minimum score to pass
    Is_Completed BOOLEAN DEFAULT true,

```



```
    UNIQUE (Job_Role_ID, Round_Number)
);
```

```
CREATE TABLE Student_Round_Result
(Result_ID SERIAL PRIMARY KEY,
Round_ID INT REFERENCES Selection_Round(Round_ID) ON DELETE CASCADE,
Application_ID INT REFERENCES Application(Application_ID) ON DELETE CASCADE,
Result VARCHAR(20) CHECK (Result IN ('Pending', 'Passed', 'Failed') ),
Score DECIMAL(5,2),
Feedback TEXT,
UNIQUE (Round_ID, Application_ID) );
```

```
CREATE TABLE Offer_Letter (
Offer_Letter_ID SERIAL PRIMARY KEY,
S_ID VARCHAR NOT NULL,
Job_Role_ID VARCHAR(10) NOT NULL,
File_Path VARCHAR(500),
File_Name VARCHAR(255),
Date_Uploaded TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
Verification_Status VARCHAR(20) DEFAULT 'Pending'
CHECK (Verification_Status IN ('Pending', 'Verified', 'Rejected')),
Reason_if_rejected TEXT,
CONSTRAINT fk_student
FOREIGN KEY (S_ID)
REFERENCES Student(S_ID)
ON DELETE CASCADE,
CONSTRAINT fk_job_role
FOREIGN KEY (Job_Role_ID)
REFERENCES Job_Role(Job_Role_ID)
ON DELETE CASCADE,
CONSTRAINT unique_student_job_offer
UNIQUE (S_ID, Job_Role_ID)
);
```

```
CREATE TABLE Placement_Coordinators (
PC_ID VARCHAR(10) PRIMARY KEY,
S_ID VARCHAR(10) NOT NULL REFERENCES Student(S_ID) ON DELETE CASCADE,
Role VARCHAR(50)
);
```

```
CREATE TABLE Company_Assigned (  
    C_ID VARCHAR(10) NOT NULL REFERENCES Company(C_ID) ON DELETE  
    CASCADE,  
    PC_ID VARCHAR(10) NOT NULL REFERENCES Placement_Coordinators(PC_ID) ON  
    DELETE CASCADE,  
    PRIMARY KEY (C_ID, PC_ID)  
);
```

```
CREATE TABLE Mentors (  
    Mentor_ID VARCHAR(10) PRIMARY KEY,  
    Mentor_Name VARCHAR(100) NOT NULL,  
    Expertise VARCHAR(100)  
);
```

```
CREATE TABLE Mentor_Student (  
    Mentor_ID VARCHAR(10) REFERENCES Mentors(Mentor_ID) ON DELETE CASCADE,  
    S_ID INT REFERENCES Student(S_ID) ON DELETE CASCADE,  
    PRIMARY KEY (Mentor_ID, S_ID)  
);
```

```
CREATE TABLE Training (  
    Training_ID VARCHAR(10) PRIMARY KEY,  
    Description TEXT,  
    Location VARCHAR(50),  
    Duration VARCHAR(20),  
    Date DATE,  
    Status VARCHAR(20) CHECK (Status IN ('Completed', 'Upcoming'))  
);
```

```
CREATE TABLE Student_Training (  
    S_ID VARCHAR(10) NOT NULL,  
    Training_ID VARCHAR(10) NOT NULL,  
    Feedback TEXT,  
    FOREIGN KEY (S_ID) REFERENCES student(S_ID) ON DELETE CASCADE,
```

```
FOREIGN KEY (Training_ID) REFERENCES Training(Training_ID) ON DELETE  
CASCADE  
);
```

```
CREATE TABLE Student_Resume (  
Resume_ID VARCHAR(10) PRIMARY KEY,  
S_ID VARCHAR(10) NOT NULL PRIMARY KEY ,  
Resume_File_Path VARCHAR(255) NOT NULL,  
Upload_Date DATE NOT NULL,  
Version_Number INT NOT NULL);
```



## **Queries**

### **Query 1**

Find all female students.

```
SELECT S_Name, Email_ID, Branch  
FROM Student  
WHERE Gender = 'Female';
```

### **Query 2**

Get students with CPI greater than 8.0

```
SELECT S_Name, Branch, CPI  
FROM Student  
WHERE CPI > 8.0  
ORDER BY CPI DESC;
```

**Query 3**

Count total number of companies

```
SELECT COUNT(*) AS Total_Companies  
FROM Company;
```

**Query 4**

List all active drives

```
SELECT Drive_ID, Drive_Year, Drive_Start_Date, Drive_Status  
FROM Drive  
WHERE Drive_Status = 'Ongoing' OR Drive_Status = 'Upcoming';
```

**Query 5**

Get alumni currently working at “Google”.

```
SELECT S_ID, Current_Job_Title FROM Alumni WHERE Current_Company = 'Google';
```

**Query 6**

Find students who graduated in 2024

```
SELECT S_Name, Branch, Graduation_Year  
FROM Student  
WHERE Graduation_Year = 2024;
```

**Query 7**

Get all companies in 'Technology' industry

```
SELECT C_Name, Website, Location  
FROM Company  
WHERE Industry_Type = 'Technology';
```

**Query 8**

Get all job roles with CTC greater than 1000000

```
SELECT Job_Role_ID, Avg_CTC_or_Stipend, Deadline  
FROM Job_Role  
WHERE Avg_CTC_or_Stipend > 1000000;
```

**Query 9**

Average CPI by branch and graduation year

```
SELECT Branch, Graduation_Year, AVG(CPI) AS Avg_CPI
FROM Student
WHERE CPI IS NOT NULL
GROUP BY Branch, Graduation_Year
ORDER BY Branch, Graduation_Year;
```

**Query 10**

Count students by branch

```
SELECT Branch, COUNT(*) AS Student_Count
FROM Student
GROUP BY Branch
ORDER BY Student_Count DESC;
```

**Query 11**

Find drives with average CTC > 10 LPA.

```
SELECT d.Drive_ID, AVG(j.Avg_CTC_or_Stipend) AS Avg_CTC
FROM Drive d
JOIN Job_Role j ON d.Drive_ID = j.Drive_ID
GROUP BY d.Drive_ID
HAVING AVG(j.Avg_CTC_or_Stipend) > 1000000;
```

**Query 12**

List placement coordinators with student names

```
SELECT pc.PC_ID, s.S_Name, pc.Role
FROM Placement_Coordinators pc
JOIN Student s ON pc.S_ID = s.S_ID;
```

**Query 13**

Get job roles with their company names

```
SELECT jr.Job_Role_ID, c.C_Name, jr.Avg_CTC_or_Stipend
FROM Job_Role jr
JOIN Company c ON jr.C_ID = c.C_ID;
```

**Query 14**

-- 19. Find job roles with academic eligibility criteria

```
SELECT jr.Job_Role_ID, jrae.Branch_name, jrae.Min_CPI_Required
FROM Job_Role jr
JOIN Job_Role_Academic_Eligibility jrae ON jr.Job_Role_ID = jrae.Job_Role_ID;
```

### **Query 15**

List students, their resumes, and applications

```
SELECT s.S_Name, s.Branch, sr.Resume_ID, a.Application_Status
FROM Student s
JOIN Student_Resume sr ON s.S_ID = sr.Student_ID
JOIN Application a ON sr.Resume_ID = a.Resume_ID;
```

### **Query 16**

Get all companies assigned to placement coordinators.

```
SELECT c.C_Name, pc.PC_ID, pc.Role
FROM Company_Assigned ca
JOIN Company c ON ca.C_ID = c.C_ID
JOIN Placement_Coordinators pc ON ca.PC_ID = pc.PC_ID;
```

### **Query 17**

Get job roles with company and drive information

```
SELECT jr.Job_Role_ID, c.C_Name, d.Drive_Year, jr.Avg_CTC_or_Stipend
FROM Job_Role jr
JOIN Company c ON jr.C_ID = c.C_ID
JOIN Drive d ON jr.Drive_ID = d.Drive_ID;
```

### **Query 18**

Get drive details with company participation

```
SELECT d.Drive_ID, dt.Type_Name, d.Drive_Start_Date,
       COUNT(DISTINCT c.C_ID) AS Company_Count
FROM Drive d
JOIN Drive_Type dt ON d.Drive_Type_ID = dt.Drive_Type_ID
JOIN Job_Role jr ON d.Drive_ID = jr.Drive_ID
JOIN Company c ON jr.C_ID = c.C_ID
GROUP BY d.Drive_ID, dt.Type_Name, d.Drive_Start_Date;
```

### **Query 19**

Find students who registered for drives but are not interested.



```

SELECT s.S_Name, r.Reason_If_Not_Interested
FROM Student s
JOIN register r ON s.S_ID = r.S_ID
WHERE r.Is_Interested = FALSE;

```

### Query 20

List job roles with their corresponding company and drive year.

```

SELECT jr.Job_Role_ID, c.C_Name, d.Drive_Year, jr.Avg_CTC_or_Stipend
FROM Job_Role jr
JOIN Company c ON jr.C_ID = c.C_ID
JOIN Drive d ON jr.Drive_ID = d.Drive_ID;

```

### Query 21

Show job roles with their required skills and locations.

```

SELECT jr.Job_Role_ID, js.Skill_Name, jl.Location
FROM Job_Role jr
LEFT JOIN Job_Role_Skill_Requirement js ON jr.Job_Role_ID = js.Job_Role_ID
LEFT JOIN Job_Role_Location jl ON jr.Job_Role_ID = jl.Job_Role_ID;

```

### Query 22

Find top 5 companies by average package with application statistics

```

WITH CompanyStats AS (
    SELECT c.C_ID, c.C_Name,
           AVG(jr.Avg_CTC_or_Stipend) AS Avg_Package,
           COUNT(DISTINCT jr.Job_Role_ID) AS Job_Roles,
           COUNT(DISTINCT a.Application_ID) AS Total_Applications,
           COUNT(DISTINCT CASE WHEN a.Application_Status = 'Selected'
                                THEN a.Application_ID END) AS Selected_Count
    FROM Company c
    JOIN Job_Role jr ON c.C_ID = jr.C_ID
    LEFT JOIN Application a ON jr.Job_Role_ID = a.Job_Role_ID
    GROUP BY c.C_ID, c.C_Name
)
SELECT C_Name, Avg_Package, Job_Roles, Total_Applications,
       Selected_Count,
       CASE
           WHEN Total_Applications > 0
           THEN ROUND(100.0 * Selected_Count / Total_Applications, 2)

```

```

        ELSE 0
    END AS Selection_Rate
FROM CompanyStats
ORDER BY Avg_Package DESC
LIMIT 5;

```

### Query 23

Rank students by applications and success rate

```

WITH StudentApplications AS (
    SELECT s.S_ID, s.S_Name, s.Branch, s.CPI,
           COUNT(a.Application_ID) AS Total_Apps,
           COUNT(CASE WHEN a.Application_Status = 'Selected'
                        THEN 1 END) AS Selected_Apps,
           COUNT(CASE WHEN a.Application_Status = 'Interviewed'
                        THEN 1 END) AS Interview_Apps
    FROM Student s
    JOIN register r ON s.S_ID = r.S_ID
    JOIN Application a ON r.Register_ID = a.Register_ID
    GROUP BY s.S_ID, s.S_Name, s.Branch, s.CPI
)
SELECT S_Name, Branch, CPI, Total_Apps, Selected_Apps, Interview_Apps,
       CASE
           WHEN Total_Apps > 0
           THEN ROUND(100.0 * Selected_Apps / Total_Apps, 2)
           ELSE 0
       END AS Success_Rate,
       RANK() OVER (PARTITION BY Branch ORDER BY Selected_Apps DESC, CPI DESC)
AS Branch_Rank
FROM StudentApplications
WHERE Total_Apps > 0
ORDER BY Success_Rate DESC, Total_Apps DESC;

```

### Query 24

Get placement success rate by branch

```
SELECT s.Branch,
       COUNT(DISTINCT CASE WHEN a.Application_Status = 'Selected'
                             THEN s.S_ID END) AS Placed_Students,
       COUNT(DISTINCT s.S_ID) AS Total_Students,
       ROUND(100.0 * COUNT(DISTINCT CASE WHEN a.Application_Status = 'Selected'
                                           THEN s.S_ID END) /
             COUNT(DISTINCT s.S_ID), 2) AS Placement_Percentage
FROM Student s
LEFT JOIN register r ON s.S_ID = r.S_ID
LEFT JOIN Application a ON r.Register_ID = a.Register_ID
GROUP BY s.Branch
ORDER BY Placement_Percentage DESC;
```

### **Query 25**

Drives with more than 10 registered students.

```
SELECT d.Drive_ID, COUNT(r.S_ID) AS Registered_Students
FROM Drive d
JOIN register r ON d.Drive_ID = r.Drive_ID
GROUP BY d.Drive_ID
HAVING COUNT(r.S_ID) > 10;
```

### **Query 26**

Find students, their registrations, and applications for a specific drive

```
SELECT s.S_Name, s.Branch, r.Registration_Date, a.Application_Status
FROM Student s
JOIN register r ON s.S_ID = r.S_ID
JOIN Drive d ON r.Drive_ID = d.Drive_ID
JOIN Application a ON r.Register_ID = a.Register_ID
WHERE d.Drive_ID = 'D_202501';
```

**Query 27**

List eligible students for each job role based on branch and CPI

```
SELECT s.S_Name, s.Branch, s.CPI, jr.Job_Role_ID, c.C_Name
FROM Student s
JOIN Job_Role_Academic_Eligibility jrae ON s.Branch = jrae.Branch_name
JOIN Job_Role jr ON jrae.Job_Role_ID = jr.Job_Role_ID
JOIN Company c ON jr.C_ID = c.C_ID
WHERE s.CPI >= jrae.Min_CPI_Required;
```

**Query 28**

List students with latest resume version who applied for jobs

```
SELECT s.S_Name, sr.Resume_ID, sr.Version_Number,
       jr.Job_Role_ID, a.Application_Status
FROM Student s
JOIN Student_Resume sr ON s.S_ID = sr.Student_ID
JOIN Application a ON sr.Resume_ID = a.Resume_ID
JOIN Job_Role jr ON a.Job_Role_ID = jr.Job_Role_ID
WHERE sr.Version_Number = (
    SELECT MAX(Version_Number)
    FROM Student_Resume
    WHERE Student_ID = s.S_ID
);
```

**Query 29**

Find students who registered but didn't apply to any job

```
SELECT s.S_Name, s.Branch, r.Registration_Date, d.Drive_ID
FROM Student s
JOIN register r ON s.S_ID = r.S_ID
JOIN Drive d ON r.Drive_ID = d.Drive_ID
LEFT JOIN Application a ON r.Register_ID = a.Register_ID
WHERE a.Application_ID IS NULL;
```

**Query 30**

List all applications with eligibility verification

```
SELECT s.S_Name, s.Branch, s.CPI, c.C_Name, jr.Job_Role_ID,
       jrae.Min_CPI_Required, a.Application_Status,
```

```

CASE
    WHEN s.CPI >= jrae.Min_CPI_Required THEN 'Eligible'
    ELSE 'Not Eligible'
END AS Eligibility_Status
FROM Application a
JOIN register r ON a.Register_ID = r.Register_ID
JOIN Student s ON r.S_ID = s.S_ID
JOIN Job_Role jr ON a.Job_Role_ID = jr.Job_Role_ID
JOIN Company c ON jr.C_ID = c.C_ID
JOIN Job_Role_Academic_Eligibility jrae
    ON jr.Job_Role_ID = jrae.Job_Role_ID AND s.Branch = jrae.Branch_name;

```

### Query 31

Compare application success rates across drives

```

SELECT d.Drive_ID, dt.Type_Name, d.Drive_Year,
    COUNT(DISTINCT a.Application_ID) AS Total_Applications,
    COUNT(DISTINCT CASE WHEN a.Application_Status = 'Applied'
        THEN a.Application_ID END) AS Applied,
    COUNT(DISTINCT CASE WHEN a.Application_Status = 'Shortlisted'
        THEN a.Application_ID END) AS Shortlisted,
    COUNT(DISTINCT CASE WHEN a.Application_Status = 'Interviewed'
        THEN a.Application_ID END) AS Interviewed,
    COUNT(DISTINCT CASE WHEN a.Application_Status = 'Selected'
        THEN a.Application_ID END) AS Selected,
    ROUND(100.0 * COUNT(DISTINCT CASE WHEN a.Application_Status = 'Selected'
        THEN a.Application_ID END) /
        NULLIF(COUNT(DISTINCT a.Application_ID), 0), 2) AS Selection_Rate
FROM Drive d
JOIN Drive_Type dt ON d.Drive_Type_ID = dt.Drive_Type_ID
JOIN Job_Role jr ON d.Drive_ID = jr.Drive_ID
LEFT JOIN Application a ON jr.Job_Role_ID = a.Job_Role_ID
GROUP BY d.Drive_ID, dt.Type_Name, d.Drive_Year
ORDER BY d.Drive_Year DESC, Selection_Rate DESC;

```

### Query 32

Find students with multiple selections

```

WITH StudentSelections AS (

```

```

SELECT s.S_ID, s.S_Name, s.Branch,
       jr.Job_Role_ID, c.C_Name, jr.Avg_CTC_or_Stipend,
       ROW_NUMBER() OVER (PARTITION BY s.S_ID
                           ORDER BY jr.Avg_CTC_or_Stipend DESC) AS selection_rank
FROM Student s
JOIN register r ON s.S_ID = r.S_ID
JOIN Application a ON r.Register_ID = a.Register_ID
JOIN Job_Role jr ON a.Job_Role_ID = jr.Job_Role_ID
JOIN Company c ON jr.C_ID = c.C_ID
WHERE a.Application_Status = 'Selected'
)
SELECT S_Name, Branch,
       MAX(CASE WHEN selection_rank = 1 THEN C_Name END) AS Best_Offer_Company,
       MAX(CASE WHEN selection_rank = 1 THEN Avg_CTC_or_Stipend END) AS
Best_Package,
       COUNT(*) AS Total_Selections,
       STRING_AGG(C_Name, ', ' ORDER BY Avg_CTC_or_Stipend DESC) AS All_Companies
FROM StudentSelections
GROUP BY S_ID, S_Name, Branch
HAVING COUNT(*) > 1
ORDER BY Best_Package DESC;

```

### Query 33

Analyze resume update patterns and application success

```

WITH ResumeStats AS (
  SELECT sr.Student_ID,
         MAX(sr.Version_Number) AS Latest_Version,
         MIN(sr.Upload_Date) AS First_Upload,
         MAX(sr.Upload_Date) AS Last_Upload,
         COUNT(DISTINCT sr.Resume_ID) AS Total_Versions
  FROM Student_Resume sr
  GROUP BY sr.Student_ID
)
SELECT s.S_Name, s.Branch, rs.Total_Versions,
       rs.First_Upload, rs.Last_Upload,
       COUNT(a.Application_ID) AS Total_Applications,
       COUNT(CASE WHEN a.Application_Status = 'Selected'
                  THEN 1 END) AS Selections,
       ROUND(AGE(rs.Last_Upload, rs.First_Upload), 0) AS Resume_Update_Period

```

```

FROM Student s
JOIN ResumeStats rs ON s.S_ID = rs.Student_ID
LEFT JOIN Student_Resume sr ON s.S_ID = sr.Student_ID
LEFT JOIN Application a ON sr.Resume_ID = a.Resume_ID
GROUP BY s.S_ID, s.S_Name, s.Branch, rs.Total_Versions,
         rs.First_Upload, rs.Last_Upload
HAVING COUNT(a.Application_ID) > 0
ORDER BY Selections DESC, Total_Versions DESC;

```

### Query 34

Find correlation between CPI and placement package

```

WITH StudentPlacements AS (
    SELECT s.S_ID, s.S_Name, s.Branch, s.CPI,
           MAX(jr.Avg_CTC_or_Stipend) AS Best_Package,
           AVG(jr.Avg_CTC_or_Stipend) AS Avg_Offered_Package,
           COUNT(DISTINCT a.Application_ID) AS Applications
    FROM Student s
    JOIN register r ON s.S_ID = r.S_ID
    JOIN Application a ON r.Register_ID = a.Register_ID
    JOIN Job_Role jr ON a.Job_Role_ID = jr.Job_Role_ID
    WHERE a.Application_Status = 'Selected'
    GROUP BY s.S_ID, s.S_Name, s.Branch, s.CPI
)
SELECT
    CASE
        WHEN CPI >= 9.0 THEN '9.0-10.0'
        WHEN CPI >= 8.0 THEN '8.0-8.9'
        WHEN CPI >= 7.0 THEN '7.0-7.9'
        ELSE 'Below 7.0'
    END AS CPI_Range,
    COUNT(*) AS Students,
    ROUND(AVG(Best_Package), 2) AS Avg_Best_Package,
    ROUND(MIN(Best_Package), 2) AS Min_Package,
    ROUND(MAX(Best_Package), 2) AS Max_Package,
    ROUND(AVG(Applications), 2) AS Avg_Applications
FROM StudentPlacements
GROUP BY CPI_Range
ORDER BY CPI_Range DESC;

```

### Query 35

Comparative analysis across branches with market insights

```
WITH branch_stats AS (
  SELECT
    s.Branch,
    COUNT(DISTINCT s.S_ID) AS total_students,
    AVG(s.CPI) AS avg_cpi,
    COUNT(DISTINCT CASE WHEN a.Application_Status = 'Selected' THEN s.S_ID END)
  AS placed_students,
    AVG(jr.Avg_CTC_or_Stipend) AS avg_package_offered,
    MAX(jr.Avg_CTC_or_Stipend) AS highest_package,
    MIN(jr.Avg_CTC_or_Stipend) AS lowest_package,
    COUNT(DISTINCT jr.C_ID) AS companies_interested,
    COUNT(DISTINCT r.Drive_ID) AS drives_participated
  FROM Student s
  LEFT JOIN registers r ON s.S_ID = r.S_ID
  LEFT JOIN Application a ON r.Register_ID = a.Register_ID
  LEFT JOIN Job_Role jr ON a.Job_Role_ID = jr.Job_Role_ID
  WHERE s.Status = 'Active'
  GROUP BY s.Branch
),
branch_rankings AS (
  SELECT
    *,
    RANK() OVER (ORDER BY avg_package_offered DESC) AS package_rank,
    RANK() OVER (ORDER BY avg_cpi DESC) AS cpi_rank
  FROM branch_stats
)
SELECT
  Branch,
  total_students,
  ROUND(avg_cpi, 2) AS avg_cpi,
  placed_students,
  ROUND(placed_students::NUMERIC / total_students * 100, 2) AS placement_percentage,
  ROUND(avg_package_offered, 2) AS avg_package_lpa,
```



```

ROUND(highest_package, 2) AS highest_package_lpa,
companies_interested
FROM branch_rankings
;

```

### Query 36

#### SKILL GAP ANALYSIS

Identifies most in-demand skills vs. student capabilities

```

WITH skill_demand AS (
  SELECT
    jrsr.Skill_Name,
    COUNT(DISTINCT jrsr.Job_Role_ID) AS roles_requiring_skill,
    COUNT(DISTINCT a.Application_ID) AS applications_for_skill,
    COUNT(DISTINCT CASE WHEN a.Application_Status = 'Selected' THEN
a.Application_ID END) AS selections_with_skill,
    AVG(jr.Avg_CTC_or_Stipend) AS avg_package_for_skill,
    COUNT(DISTINCT jr.C_ID) AS companies_seeking_skill
  FROM Job_Role_Skill_Requirement jrsr
  JOIN Job_Role jr ON jrsr.Job_Role_ID = jr.Job_Role_ID
  LEFT JOIN Application a ON jr.Job_Role_ID = a.Job_Role_ID
  GROUP BY jrsr.Skill_Name
)
SELECT
  Skill_Name,
  roles_requiring_skill,fun
companies_seeking_skill,
applications_for_skill,
selections_with_skill,
ROUND(selections_with_skill::NUMERIC / NULLIF(applications_for_skill, 0) * 100, 2) AS
success_rate_percentage,
ROUND(avg_package_for_skill, 2) AS avg_package_lpa,
CASE
  WHEN roles_requiring_skill >= 10 THEN 'Critical Skill'
  WHEN roles_requiring_skill >= 5 THEN 'High Demand'
  WHEN roles_requiring_skill >= 2 THEN 'Moderate Demand'
  ELSE 'Niche Skill'
END AS demand_category,
ROUND(roles_requiring_skill * avg_package_for_skill, 2) AS skill_market_value

```

```
FROM skill_demand
WHERE roles_requiring_skill > 0
ORDER BY skill_market_value DESC, roles_requiring_skill DESC
LIMIT 20;
```

## Triggers

1

trg\_prevent\_duplicate\_application

BEFORE INSERT on Application: Prevents a student from applying to the same job role more than once.

sql

```
CREATE OR REPLACE
```

```
FUNCTION prevent_duplicate_application() RETURNS TRIGGER AS $$
```

```
BEGIN IF EXISTS ( SELECT 1 FROM Application a
```

```
JOIN registers r ON a.Register_ID = r.Register_ID
```

```
WHERE r.Register_ID = NEW.Register_ID AND a.Job_Role_ID = NEW.Job_Role_ID )
```

```
THEN RAISE EXCEPTION 'Student has already applied for this job role (Register ID: %, Job  
Role ID: %)', NEW.Register_ID, NEW.Job_Role_ID; END IF; RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_prevent_duplicate_application BEFORE INSERT ON Application FOR  
EACH ROW EXECUTE FUNCTION prevent_duplicate_application();
```

Trigger Update Company Last Visit Information

```
CREATE OR REPLACE FUNCTION update_company_last_visit()
```

```
RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
UPDATE Company
```

```
SET Last_Visit_Year = EXTRACT(YEAR FROM NEW.Drive_Start_Date)
```

```
WHERE C_ID IN (
```

```
SELECT DISTINCT C_ID
```

```
FROM Job_Role
```

```
WHERE Drive_ID = NEW.Drive_ID
```

```
);
```

```
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_update_company_last_visit
AFTER INSERT ON Drive
FOR EACH ROW
EXECUTE FUNCTION update_company_last_visit();
```

Trigger :Validate Eligibility Before Application

```
CREATE OR REPLACE FUNCTION validate_application_eligibility()
RETURNS TRIGGER AS $$
DECLARE
    v_s_id VARCHAR(10);
    v_job_role_id VARCHAR(10);
    v_is_eligible BOOLEAN;
BEGIN
    -- Get student ID and job role ID
    SELECT r.S_ID INTO v_s_id
    FROM register r
    WHERE r.Register_ID = NEW.Register_ID;

    SELECT NEW.Job_Role_ID INTO v_job_role_id;

    -- Check eligibility
    v_is_eligible := check_student_eligibility(v_s_id, v_job_role_id);

    IF NOT v_is_eligible THEN
        RAISE EXCEPTION 'Student does not meet eligibility criteria for this job role';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_validate_application_eligibility
BEFORE INSERT ON Application
FOR EACH ROW
EXECUTE FUNCTION validate_application_eligibility();
```

Automatically move graduated students to Alumni table.

```
CREATE OR REPLACE FUNCTION insert_alumni()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.Status = 'Graduated' THEN
        INSERT INTO Alumni (S_ID, Current_Job_Title, Current_Company)
        VALUES (NEW.S_ID, 'Not Updated', 'Not Updated');
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_alumni_insert
AFTER UPDATE ON Student
FOR EACH ROW
EXECUTE FUNCTION insert_alumni();
```

## Stored Functions

### F1

check\_student\_eligibility

Returns BOOLEAN based on whether a student meets the minimum CPI for a job role.

Sql

```
CREATE OR REPLACE FUNCTION check_student_eligibility(student_id VARCHAR,
job_role_id VARCHAR) RETURNS BOOLEAN AS $$
DECLARE v_student_cpi NUMERIC(3, 2);
v_min_cpi NUMERIC(3, 2);
BEGIN SELECT s.CPI INTO v_student_cpi FROM Student s WHERE s.S_ID = student_id;
SELECT jrae.Min_CPI_Required INTO v_min_cpi FROM Job_Role_Academic_Eligibility jrae
WHERE jrae.Job_Role_ID = job_role_id; IF v_min_cpi IS NULL OR v_student_cpi >=
v_min_cpi THEN RETURN TRUE;
ELSE RETURN FALSE;
END IF;
```

```
END;  
$$ LANGUAGE plpgsql;
```

## **F2**

get\_latest\_resume\_id

Returns the Resume\_ID (VARCHAR) for the latest version uploaded by a student.

Sql

```
CREATE OR REPLACE FUNCTION get_latest_resume_id(student_id VARCHAR)  
RETURNS VARCHAR AS $$  
DECLARE v_resume_id VARCHAR(10);  
BEGIN SELECT Resume_ID INTO v_resume_id FROM Student_Resume WHERE S_ID =  
student_id  
ORDER BY Version_Number DESC,  
Upload_Date DESC LIMIT 1;  
RETURN v_resume_id; END;  
$$ LANGUAGE plpgsql;
```

## **F3**

calculate\_placement\_rate

Calculates the overall placement percentage (NUMERIC) for a given branch.

sql

```
CREATE OR REPLACE FUNCTION calculate_placement_rate(branch_name VARCHAR)  
RETURNS NUMERIC AS $$  
DECLARE v_placed_count INT; v_total_students INT;  
BEGIN SELECT COUNT(DISTINCT s.S_ID) INTO v_total_students FROM Student s  
WHERE s.Branch = branch_name;  
SELECT COUNT(DISTINCT s.S_ID) INTO v_placed_count FROM Student s JOIN registers r  
ON s.S_ID = r.S_ID  
JOIN Application a ON r.Register_ID = a.Register_ID  
WHERE s.Branch = branch_name AND a.Application_Status = 'Selected';  
IF v_total_students = 0 THEN RETURN 0.00;  
ELSE RETURN ROUND((v_placed_count::NUMERIC / v_total_students) * 100, 2); END IF;  
END;  
$$ LANGUAGE plpgsql;
```

## **F4**

get\_drive\_avg\_package

Returns the average CTC/Stipend (NUMERIC) offered across all job roles in a specific drive.

sql

```
CREATE OR REPLACE FUNCTION get_drive_avg_package(drive_id VARCHAR) RETURNS
NUMERIC AS $$ DECLARE v_avg_ctc NUMERIC;
BEGIN SELECT AVG(Avg_CTC_or_Stipend) INTO v_avg_ctc FROM Job_Role
WHERE Drive_ID = drive_id; RETURN COALESCE(v_avg_ctc, 0); END;
$$ LANGUAGE plpgsql;
```

## **F5**

get\_student\_mentor\_name

Returns the name (VARCHAR) of the primary mentor for a given student ID.

Sql

```
CREATE OR REPLACE FUNCTION get_student_mentor_name(student_id VARCHAR)
RETURNS VARCHAR AS $$
DECLARE v_mentor_name VARCHAR(100); BEGIN SELECT m.Mentor_Name INTO
v_mentor_name
FROM Mentor_Student ms
JOIN Mentors m ON ms.Mentor_ID = m.Mentor_ID WHERE ms.S_ID = student_id LIMIT 1;
RETURN COALESCE(v_mentor_name, 'No Mentor Assign
```

## **Procedures**

### **P1**

increment\_hired\_count

Increments a company's Last\_Hired\_Count when a student is selected.

sql

```
CREATE OR REPLACE PROCEDURE increment_hired_count(cid VARCHAR)
LANGUAGE plpgsql AS $$
BEGIN UPDATE Company SET Last_Hired_Count = COALESCE(Last_Hired_Count, 0) + 1
WHERE C_ID = cid; END; $$;
```

### **P2**

update\_drive\_status

Changes the status of past drives to 'Completed'.

sql

```
CREATE OR REPLACE PROCEDURE update_drive_status()
LANGUAGE plpgsql AS $$
BEGIN UPDATE Drive SET Drive_Status = 'Completed'
WHERE Drive_End_Date < CURRENT_DATE AND Drive_Status != 'Completed'; END; $$;
CALL update_drive_status();
```

### **P3**

process\_application\_selection

Marks all other applications as 'Rejected' for a student once one application is marked 'Selected'.

sql

```
CREATE OR REPLACE PROCEDURE process_application_selection(selected_app_id INT)
LANGUAGE plpgsql AS $$
DECLARE v_s_id VARCHAR(10);
BEGIN SELECT r.S_ID INTO v_s_id FROM Application a
JOIN registers r ON a.Register_ID = r.Register_ID WHERE a.Application_ID =
selected_app_id; UPDATE Application SET Application_Status = 'Rejected'
WHERE Register_ID IN (SELECT Register_ID FROM registers
WHERE S_ID = v_s_id) AND Application_Status != 'Selected' AND Application_ID !=
selected_app_id; END; $$;
```

### **P4**

add\_new\_job\_role

Inserts a new job role, ensuring the company and drive exist.

sql

```
CREATE OR REPLACE PROCEDURE add_new_job_role( jrid VARCHAR, driveid
VARCHAR, cid VARCHAR, title VARCHAR, ctc NUMERIC)
LANGUAGE plpgsql AS $$
BEGIN IF NOT EXISTS (SELECT 1 FROM Company WHERE C_ID = cid) OR NOT EXISTS
(SELECT 1 FROM Drive WHERE Drive_ID = driveid)
THEN RAISE EXCEPTION 'Company or Drive ID does not exist.'; END IF;
INSERT INTO Job_Role (Job_Role_ID, Drive_ID, C_ID, Title, Avg_CTC_or_Stipend)
VALUES (jrid, driveid, cid, title, ctc); END; $$;
```

**P5**

update\_student\_status\_to\_placed

Updates a student's Status to 'Placed' (assuming 'Placed' is a valid status) if they have a 'Selected' application.

Sql

```
CREATE OR REPLACE PROCEDURE update_student_status_to_placed(student_id
VARCHAR)
LANGUAGE plpgsql AS $$
BEGIN UPDATE Student SET Status = 'Placed'
WHERE S_ID = student_id AND EXISTS ( SELECT 1 FROM Application a JOIN registers r
ON a.Register_ID = r.Register_ID WHERE r.S_ID = student_id AND a.Application_Status =
'Selected' ); END; $$;
```

**P6**

Update drive status automatically after end date.

```
CREATE OR REPLACE PROCEDURE update_drive_status()
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE Drive
    SET Drive_Status = 'Completed'
    WHERE Drive_End_Date < CURRENT_DATE AND Drive_Status != 'Completed';
END;
$$;
```

**P7**

Increment company's last hired count.

```
CREATE OR REPLACE PROCEDURE increment_hired_count(cid VARCHAR)

LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE Company
```



```
        SET Last_Hired_Count = COALESCE(Last_Hired_Count, 0) + 1
        WHERE C_ID = cid;
END;
$$;
CALL increment_hired_count('C_1001');
```